CLAUDE CERTIFIED ARCHITECT - FOUNDATIONS | EXAM CODE CCAR-F

Mock Examination 04

Claude Certified Architect - Foundations
60 scenario-based questions | 120 minutes

Scenarios assessed in this paper: Scenario 1 - Customer Support Resolution Agent; Scenario 2 - Code Generation with Claude Code; Scenario 4 - Developer Productivity with Claude; Scenario 6 - Structured Data Extraction

Format: multiple-choice and multiple-response; every question states how many answers to select.

Contents: instructions and blueprint | candidate answer sheet | 60 questions | quick answer key and score tracker | detailed explanations for every option | domain-level review guide | revision checklist.

Status: ORIGINAL PRACTICE MATERIAL. This is not an official Anthropic examination paper and contains no confidential exam content. It was written against the publicly available Claude Certified Architect - Foundations Exam Guide (v1.0, effective July 2026).

Contents

1. Instructions and exam conditions 3

2. What is verified, what is supplied, and what is original 3

3. Exam blueprint for this paper 4

Candidate answer sheet 5

4. Examination - Questions 1 to 60 6

Scenario 1: Customer Support Resolution Agent (Questions 1-15) . . . 6

Scenario 2: Code Generation with Claude Code (Questions 16-28) . . . 10

Scenario 4: Developer Productivity with Claude (Questions 29-43) . . . 13

Scenario 6: Structured Data Extraction (Questions 44-60) . . . 16

5. Quick answer key and score tracker 21

6. Detailed answer key and explanations 22

7. Domain-level performance review guide 46

Domain 1 - Agentic Architecture & Orchestration (27% official weight; 16 questions here) . . . 46

Domain 2 - Tool Design & MCP Integration (18% official weight; 11 questions here) . . . 46

Domain 3 - Claude Code Configuration & Workflows (20% official weight; 12 questions here) . . . 46

Domain 4 - Prompt Engineering & Structured Output (20% official weight; 12 questions here) . . . 46

Domain 5 - Context Management & Reliability (15% official weight; 9 questions here) . . . 47

8. Final revision checklist 48

1. Instructions and exam conditions

Time limit: 120 minutes for 60 questions (an average of 2 minutes per question). Work in one sitting, without notes, documentation, or AI assistance, to get a realistic read on your readiness.

• Every question is anchored to one of the production scenarios in this paper. Read the scenario description at the start of each scenario section; questions are otherwise self-contained.
• Each question states **how many answers to select**. For 'Select ONE', exactly one option is best. For 'Select exactly TWO/THREE', all selected options must be correct; there is no partial credit in this practice format.
• Choose the *best* answer for the stated constraints, not an answer that could merely work. Where a question asks for the FIRST, SAFEST, or MOST RELIABLE step, that qualifier is part of the question.
• Attempt every question. Unanswered questions score zero. There is no 'all of the above' or 'none of the above'.
• Do not look at the answer key until you have finished. The key, explanations, and review guide begin after Question 60.

2. What is verified, what is supplied, and what is original

Category	What it covers
Verified official information	From the official Exam Guide (CCAR-F v1.0): 60 items; multiple-choice and multiple-response items that state how many responses to select; 4 scenarios drawn from a bank of 6; 120-minute limit; scaled passing score of 720 on a 100-1,000 scale; score report with percent-correct by domain; domain weights 27/18/20/20/15%; the 31 task statements and the in-scope/out-of-scope lists.
Scenario descriptions	The six production scenarios are quoted from the official guide; the extra topic lists you were given (e.g., idempotency, structured handoffs, schema design) guided question themes. Scenario text shown above each question group is the official description.
Original practice content	All questions, options, explanations, and review guidance in this file were written for this pack. They are not official questions, are not copied from commercial practice exams or leaked banks, and may differ in difficulty from the real exam.
Supplemental knowledge	A few items use general engineering knowledge to make a scenario realistic (for example CI/CD terminology or least-privilege shell access). They are flagged 'Supplemental' in the explanation and are not confirmed exam objectives.

A raw percentage on this paper is **not** a scaled score. The real exam reports a scaled score (100-1,000) with a cut score of 720 derived from a formal standard-setting study; percent-correct by domain is shown for information only. Doing well on these papers does not guarantee a pass.

3. Exam blueprint for this paper

Domain	Official weight	Questions in this paper
1. Agentic Architecture & Orchestration	27%	16
2. Tool Design & MCP Integration	18%	11
3. Claude Code Configuration & Workflows	20%	12
4. Prompt Engineering & Structured Output	20%	12
5. Context Management & Reliability	15%	9
Total	**100%**	**60**

Question counts are the closest integer allocation of the official weights to 60 questions. Each question has exactly one primary domain; secondary domains appear in the answer key where a question also touches another domain.

Scenario	Questions	Numbers
Scenario 1 - Customer Support Resolution Agent	15	1-15
Scenario 2 - Code Generation with Claude Code	13	16-28
Scenario 4 - Developer Productivity with Claude	15	29-43
Scenario 6 - Structured Data Extraction	17	44-60

Design targets (not claims about the real exam): about 20% foundational, 55% intermediate, 25% challenging. In this paper: 12 foundational, 33 intermediate, 15 challenging. Response types: 54 single-answer, 5 select-two, 1 select-three.

Candidate answer sheet

Mark your answers here while taking the paper. Cells that do not exist for a question are greyed out. Where a question says 'Select exactly TWO/THREE', mark exactly that many letters. Do not turn to the answer key until you have finished all 60 questions (or the 120 minutes have elapsed).

Name: _____ Date: _____ Start time: _____ End time: _____

No.	Pick	A	B	C	D	E	F
1	1	(A)	(B)	(C)	(D)		
2	1	(A)	(B)	(C)	(D)		
3	1	(A)	(B)	(C)	(D)		
4	1	(A)	(B)	(C)	(D)		
5	1	(A)	(B)	(C)	(D)		
6	1	(A)	(B)	(C)	(D)		
7	1	(A)	(B)	(C)	(D)		
8	1	(A)	(B)	(C)	(D)		
9	1	(A)	(B)	(C)	(D)		
10	1	(A)	(B)	(C)	(D)		
11	1	(A)	(B)	(C)	(D)		
12	2	(A)	(B)	(C)	(D)	(E)	
13	2	(A)	(B)	(C)	(D)	(E)	
14	1	(A)	(B)	(C)	(D)		
15	1	(A)	(B)	(C)	(D)		
16	1	(A)	(B)	(C)	(D)		
17	1	(A)	(B)	(C)	(D)		
18	1	(A)	(B)	(C)	(D)		
19	1	(A)	(B)	(C)	(D)		
20	1	(A)	(B)	(C)	(D)		
21	1	(A)	(B)	(C)	(D)		
22	1	(A)	(B)	(C)	(D)		
23	1	(A)	(B)	(C)	(D)		
24	2	(A)	(B)	(C)	(D)	(E)	
25	1	(A)	(B)	(C)	(D)		
26	2	(A)	(B)	(C)	(D)	(E)	
27	1	(A)	(B)	(C)	(D)		
28	1	(A)	(B)	(C)	(D)		
29	3	(A)	(B)	(C)	(D)	(E)	(F)
30	1	(A)	(B)	(C)	(D)		

No.	Pick	A	B	C	D	E	F
31	1	(A)	(B)	(C)	(D)		
32	1	(A)	(B)	(C)	(D)		
33	1	(A)	(B)	(C)	(D)		
34	1	(A)	(B)	(C)	(D)		
35	1	(A)	(B)	(C)	(D)		
36	1	(A)	(B)	(C)	(D)		
37	1	(A)	(B)	(C)	(D)		
38	1	(A)	(B)	(C)	(D)		
39	1	(A)	(B)	(C)	(D)		
40	1	(A)	(B)	(C)	(D)		
41	1	(A)	(B)	(C)	(D)		
42	1	(A)	(B)	(C)	(D)		
43	1	(A)	(B)	(C)	(D)		
44	1	(A)	(B)	(C)	(D)		
45	1	(A)	(B)	(C)	(D)		
46	1	(A)	(B)	(C)	(D)		
47	1	(A)	(B)	(C)	(D)		
48	1	(A)	(B)	(C)	(D)		
49	1	(A)	(B)	(C)	(D)		
50	1	(A)	(B)	(C)	(D)		
51	1	(A)	(B)	(C)	(D)		
52	1	(A)	(B)	(C)	(D)		
53	2	(A)	(B)	(C)	(D)	(E)	
54	1	(A)	(B)	(C)	(D)		
55	1	(A)	(B)	(C)	(D)		
56	1	(A)	(B)	(C)	(D)		
57	1	(A)	(B)	(C)	(D)		
58	1	(A)	(B)	(C)	(D)		
59	1	(A)	(B)	(C)	(D)		
60	1	(A)	(B)	(C)	(D)		

4. Examination - Questions 1 to 60

Time allowed: 120 minutes. Start the clock now.

Scenario 1: Customer Support Resolution Agent (Questions 1-15)

You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (get_customer, lookup_order, process_refund, escalate_to_human). Your target is 80%+ first-contact resolution while knowing when to escalate.

Question 1

Select ONE answer.

Audit logs show some process_refund calls where the agent supplied an order ID and amount that it never retrieved with lookup_order, for example amounts copied from the customer's message. Some refunds exceeded what the order allowed. Which control best prevents this class of error?

A. A requirement that the agent writes a short justification for each refund in the tool call, which a reviewer can read in the logs the following day.

B. A gate that rejects process_refund unless a successful lookup_order result for that order ID exists in the session and the amount is within the refundable balance.

C. A system prompt rule instructing the agent to look up the order, confirm the amount with the customer in the conversation, and only then issue the refund through the tool.

D. A lower per-call maximum for process_refund so that any single mistaken amount stays small, with retries allowed when the refund is below the limit.

Question 2

Select ONE answer.

A team adds few-shot examples and stronger wording to the prompt to enforce 'verify identity before discussing billing disputes'. Skipped verification falls from 8% to 4% of cases. A product manager says 4% is good enough. What is the most defensible reasoning?

A. Keep adding examples until skipped verification is below 1%, then re-evaluate, since each new example should cut the failure rate further.

B. Switch to a larger model, because a more capable model will follow the verification rule in every case.

C. Because skipped verification exposes data and enables wrong financial actions, add a programmatic prerequisite gate rather than relying on prompts.

D. Add an end-of-day report that lists conversations where verification was skipped so that a manager can contact the affected customers.

Question 3

Select ONE answer.

A developer's loop executes the tool the model requests but then sends the next request containing only the original conversation, not the tool call or its result. The agent repeatedly calls lookup_order for the same order until the iteration cap is hit. What is wrong?

A. The model needs a stronger instruction not to repeat tool calls that it has already made earlier in the conversation.

B. The tool should return its result as a system message so the model gives it higher priority on the next turn.

C. The assistant's tool_use message and the matching tool_result must be appended to the conversation before the next request so the model can use the new information.

D. The iteration cap is too low, and raising it to a much higher number will let the model eventually proceed without needing the earlier result at all, which is the cheapest fix.

Question 4

Select ONE answer.

The agent must escalate a disputed $1,250 charge to a billing specialist who cannot see the chat. Which handoff payload is best?

A. Customer ID C-2210; disputed charge $1,250 on order #7731 (duplicate billing on 2 Mar); root cause: payment captured twice; refund exceeds agent limit; recommended: reverse the duplicate capture; customer expects resolution within 5 days.

B. Customer: Priya M. Issue: billing problem with a recent order. The customer is upset about a large amount and wants it fixed quickly. Please review the full chat for details.

C. The complete transcript of all 42 messages and the raw JSON from every tool call, attached as a text file for the specialist to read.

D. Customer ID C-2210; order #7731; the customer is angry and has contacted us twice before; please handle with care and call them as soon as you can today.

Question 5

Select ONE answer.

A hook blocks any single refund above $500 and redirects to human escalation. In testing, the agent responds to a $900 request by issuing two refunds of $450 for the same order, each below the threshold. How should the control be strengthened?

A. Add a prompt instruction telling the agent never to split a refund into several smaller payments in order to avoid the review threshold, and to explain why.

B. Make the check cumulative: evaluate the total refunded for the order including the new amount, and block and redirect when that total would exceed the threshold.

C. Require the agent to wait 60 seconds between refund calls so that a person has time to notice unusual patterns.

D. Reduce the per-call threshold to $250 so that splitting becomes less attractive to the agent.

Question 6

Select ONE answer.

What does fork_session provide in an agent workflow?

A. A faster version of the same conversation that runs with a cheaper model and discards earlier tool results to save time.

B. A way to run the same tool call in parallel across several customer accounts and merge the results into the main session.

C. Independent branches of a conversation created from a shared baseline, so different approaches can be explored without affecting each other.

D. A permanent backup of the session transcript that is restored automatically if the agent crashes during a tool call.

Question 7

Select ONE answer.

A support team wants the agent to handle returns, billing disputes, and account issues, with many ambiguous messages. One proposal is a decision tree in code that chooses the sequence of tools by request category. Another is to let Claude choose tools in the loop and use hooks for hard rules. Which is more appropriate, and why?

A. Let Claude choose tools within the loop because the right next action depends on context, and enforce the few non-negotiable rules with hooks and gates.

B. Let Claude choose tools and rely on the system prompt alone for rules such as refund limits and verification.

C. Use the decision tree for everything, because predictable sequences are always more reliable than model reasoning on ambiguous requests, and the team can test every path in advance.

D. Use the decision tree for category detection and the model only for writing the final reply to the customer.

Question 8

Select ONE answer.

To make sure the agent always calls a tool, a developer sets tool_choice to 'any' on every turn. Now the agent cannot simply ask 'Could you tell me your order number?' and instead makes tool calls with made-up arguments. The agent must still be able to answer in plain text when no tool is needed. What is the better configuration?

A. Keep 'any' on every turn and add a tool named ask_customer that wraps clarifying questions as a tool call, so that every turn still produces a tool call.

B. Use 'auto' for normal turns so the model can ask questions, and force a specific tool only at a step where it must run first and its inputs are available.

C. Remove tool_choice entirely and tell the model in the prompt to always use a tool, never plain text.

D. Force get_customer on every turn so that identity lookup always happens before any reply.

Question 9

Select ONE answer.

Two customer tools are often confused, get_customer (by email or phone) and lookup_order (by order number). Which addition to their descriptions is most helpful for reliable selection?

A. A sentence at the end of each description saying that the tool is important and should be used whenever it might be relevant.

B. A longer list of every field that the tool's backend database stores, whether or not the tool returns it.

C. For each tool, the accepted input formats, example requests, and an explicit statement of when to use it versus the other tool.

D. The internal service name and team owner for each tool so that the model understands who maintains them.

Question 10

Select ONE answer.

The agent often misapplies refund rules because it doesn't know which return categories and policy versions exist, so it calls a search_policies tool several times per conversation to discover them. How can you reduce this exploratory calling?

A. Add three more search tools with different parameters and similar names so that the agent can find the right policy documents in fewer attempts.

B. Expose the catalog of return categories and policy versions as an MCP resource so the agent can see what exists without repeated tool calls.

C. Ask the customer to state which policy applies so the agent does not have to look for it.

D. Remove search_policies and let the agent answer from its general knowledge of common return policies.

Question 11

Select ONE answer.

Customers write 'I was charged for something I returned.' The agent sometimes treats this as a return and sometimes as a billing dispute, using different tools and policies each time. Which prompt improvement best makes classification consistent?

A. Ask the agent to handle every ambiguous message as both categories simultaneously and report two outcomes.

B. Tell the agent to choose whichever category it thinks is more probable for each message, and to try to stay consistent in its choices across the whole day of conversations.

C. Require the customer to choose a category from a menu before the conversation starts.

D. Provide explicit criteria with boundary rules, for example: refund already issued but charge still posted means billing dispute; item not yet returned means return.

Question 12

Select exactly TWO answers (choose 2).

Select the TWO conditions that are reliable triggers for escalating a support case to a human.

A. A sentiment score for the customer's last message falls below a set threshold.

B. The customer's request involves a situation the written policy does not cover or leaves ambiguous.

C. The conversation has exceeded ten turns, regardless of progress.

D. The agent cannot make meaningful progress after its tools keep failing or returning inconclusive results.

E. The model reports that its confidence in its answer is below 7 out of 10.

Question 13

Select exactly TWO answers (choose 2).

Select the TWO behaviors that are correct for the support agent.

A. When several accounts match, proceed with the one with the most recent activity and tell the customer which account was used.

B. When get_customer returns several accounts matching a name, ask for an additional identifier before proceeding.

C. When the agent feels unsure, ask the customer to rate how satisfied they are with the answers so far.

D. When a customer sounds frustrated but has not asked for a person, escalate immediately to be safe.

E. When a customer explicitly asks for a human, escalate right away without first investigating the problem.

Question 14

Select ONE answer.

After 30 turns, the agent tells a customer 'your refund of $48 will arrive in 2 days.' The customer was told earlier, from order data, that the refund was $84.50 and to expect it in 5 business days. The conversation summary retained only 'refund approved'. What design best prevents this?

A. Ask the model to re-derive the refund amount from the conversation each time it mentions it.

B. Keep a persistent case-facts block of amounts, dates, and customer expectations from tool results, included in every prompt outside the summarized history.

C. Switch to a model with a bigger context window so that no summarization is ever needed.

D. Summarize more often, say every five turns, so that each summary is closer in time to the facts it describes, and tell the summarizer to keep all numbers.

Question 15

Select ONE answer.

A customer writes 'Please cancel it.' The context could mean cancelling a pending order or closing the account, and account closure is irreversible. What should the agent do?

A. Cancel the pending order, since it is the more likely meaning and the less destructive of the two.

B. Escalate immediately because ambiguous cancellation requests are always too risky for an automated agent.

C. Ask a short clarifying question to establish whether the customer means the order or the account before taking any action.

D. Close the account and tell the customer how to reopen it if that was a mistake.

Scenario 2: Code Generation with Claude Code (Questions 16-28)

You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution.

Question 16

Select ONE answer.

After analyzing a payments module in Claude Code, you want to compare two testing strategies, one with heavy mocking and one using an in-memory database, both starting from the same codebase analysis. How should you set this up?

A. Ask Claude to implement both strategies in a single conversation, one after the other, and compare the results at the end.

B. Resume the same named session in two terminals and let each pursue a different strategy in the same history.

C. Use fork_session to create two branches from the shared analysis and develop each strategy independently.

D. Start two brand-new sessions and ask each to redo the codebase analysis before building its strategy.

Question 17

Select ONE answer.

You need to list every Storybook file (*.stories.tsx) across a monorepo with around 80 packages. Which approach is best in Claude Code?

A. Use Grep to search for the text 'stories' in every file and treat each matching file as a Storybook file.

B. Use Read on the root directory and ask Claude to scan the listing by eye for matching file names.

C. Use Bash to run find and cat on all files and then ask Claude to filter the combined output.

D. Use Glob with a pattern such as **/*.stories.tsx to match file paths directly.

Question 18

Select ONE answer.

A developer's response style preferences (short answers, no emojis) should apply in all of their projects, but the team's coding standards should apply to everyone working in this repository. Where should each go?

A. Both in the user-level ~/.claude/CLAUDE.md, so that the developer's settings follow them everywhere.

B. Preferences in a directory-level CLAUDE.md in each package, and standards in the user-level file.

C. Both in the project-level CLAUDE.md, so that the whole team gets the same style of answers as well.

D. Preferences in the user-level ~/.claude/CLAUDE.md, and team standards in the project-level CLAUDE.md committed to the repository.

Question 19

Select ONE answer.

A developer must replace a deprecated API across about 400 files. Finding all usages produces thousands of lines of search output, and the developer wants to design the migration before editing anything. Which workflow is best?

A. Run all searches in the main conversation and begin editing files as they appear in the results, fixing each usage as soon as it is found.

B. Ask Claude to make the replacement in all 400 files in one direct-execution request with a one-line description.

C. Use plan mode, have the Explore subagent do the search and return a summary of usage patterns, then design the migration and execute it.

D. Skip the search and replace the API in the files the developer remembers using it.

Question 20

Select ONE answer.

You are creating a project skill, /pr-description, that should read a diff file already saved in the repository, write a description file, and never run shell commands. It must run in an isolated context to keep the main session clean, and prompt developers for the PR number if omitted. Which SKILL.md frontmatter fits?

A. context: fork, allowed-tools: Bash, Read, Write, and argument-hint: [pr-number].

B. context: fork, allowed-tools limited to Read and Write, and argument-hint: [pr-number].

C. context: fork, argument-hint: [pr-number], and a note in the body that Bash must not be used.

D. allowed-tools limited to Read and Write, argument-hint: [pr-number], and no context setting.

Question 21

Select ONE answer.

Which of the following belongs in a team's project-level CLAUDE.md rather than in an individual's ~/.claude/CLAUDE.md?

A. The developer's habit of asking Claude to open files in a particular editor on their machine.

B. A developer's preference for how long Claude's explanations are in their own terminal sessions.

C. The team's rule that all new services must include structured logging and a health-check endpoint.

D. A reminder for the developer to review their own side projects on Friday afternoon.

Question 22

Select ONE answer.

Terraform modules in your repository live both under terraform/ and in several infra/ subfolders, and all .tf files must follow the same module conventions. A rule is currently scoped to paths: ["terraform/**/*"], so edits in infra/ ignore it. What is the best fix?

A. Delete the paths field so the rule loads on every request and covers every directory.

B. Move the Terraform conventions into the root CLAUDE.md so that they apply everywhere.

C. Duplicate the rule file for each infra/ subfolder with a path matching that subfolder.

D. Scope the rule with a glob by file type, such as paths: ["**/*.tf"], so it applies wherever .tf files are edited.

Question 23

Select ONE answer.

A senior engineer says, 'Always use plan mode first, it is the professional approach.' Another says, 'Plan mode is overhead, always execute directly.' Which view best reflects the guidance for choosing?

A. Use plan mode for every task over ten lines of code, since planning time is always cheaper than any rework, and the extra step guards against surprises.

B. Use plan mode for large, multi-file, or architectural tasks with several valid approaches, and direct execution for well-scoped changes with a clear cause.

C. Use plan mode only when a task touches more than one file, and never for changes inside a single file.

D. Use direct execution for all tasks and switch to plan mode only when the first attempt fails twice.

Question 24

Select exactly TWO answers (choose 2).

Select the TWO statements about sharing Claude Code configuration that are correct.

A. Skills in ~/.claude/skills/ are shared with the team whenever the project is pulled.

B. Instructions in the project-level CLAUDE.md are shared with teammates through version control.

C. Instructions in ~/.claude/CLAUDE.md apply only to that user and are not shared with teammates.

D. MCP servers defined in a developer's ~/.claude.json are automatically available to every teammate.

E. Slash commands in ~/.claude/commands/ are automatically available to everyone who clones the repository.

Question 25

Select ONE answer.

You want testing conventions to load only for test files in every package, while keeping the root CLAUDE.md small. A teammate proposes putting the conventions in a file imported with @import from the root CLAUDE.md. Why is a path-scoped rule better here?

A. An imported file loads along with CLAUDE.md regardless of the file being edited, whereas a path-scoped rule loads only when matching test files are involved.

B. Path-scoped rules can contain more text than imported files, so the test conventions fit in one place.

C. Path-scoped rules are applied to teammates only if they run a slash command first.

D. Imported files are ignored by Claude Code when they are referenced from the root CLAUDE.md, so conventions placed there would never take effect in any session.

Question 26

Select exactly TWO answers (choose 2).

Claude Code writes pull request descriptions in varying formats and misjudges how to describe refactor-only PRs that change no behavior. Select the TWO techniques most likely to improve consistency and judgment.

A. Ask Claude to produce each description twice and choose the longer one.

B. Provide thirty examples drawn from last year's PRs so that the model has seen as many variations as possible.

C. Write the format requirements in capital letters at the top of the prompt for emphasis.

D. Include an ambiguous refactor-only example with a short explanation of why it is described the way it is.

E. Provide two to four examples of well-formed descriptions showing the desired sections (summary, risk, test plan).

Question 27

Select ONE answer.

You maintain a scratchpad file during a long Claude Code exploration. What should it contain to be most useful for later questions?

A. General notes about typical architectures of similar systems, to give Claude background knowledge.

B. A running word count and timestamps for each action so that progress can be measured.

C. The full raw output of every search and file read performed so far.

D. Key findings: specific class and file names, call paths, decisions made, and open questions.

Question 28

Select ONE answer.

A developer pastes a 3,000-line CI log into Claude Code. The single failing assertion is buried around line 1,400, and Claude's analysis misses it and discusses unrelated warnings. What is the best way to present the input?

A. Paste the log in reverse order so the end of the file appears first.

B. Ask Claude to read the log slowly and carefully before answering.

C. Paste the log twice in the same message so that the failing assertion appears in two places and is therefore more likely to be noticed by the model.

D. Lead with a short summary of the failing test and the question, trim the log to relevant sections, and organize the rest under clear headers.

Scenario 4: Developer Productivity with Claude (Questions 29-43)

You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

Question 29

Select exactly THREE answers (choose 3).

A team is configuring a coordinator and subagents with the Claude Agent SDK. Select the THREE statements that are correct.

A. The coordinator's allowedTools must include "Task" for it to be able to spawn subagents.

B. Subagents can see the coordinator's full conversation history, so passing results in the prompt is optional.

C. Context needed by a subagent must be provided explicitly in its prompt because it does not inherit the coordinator's history.

D. Subagents share a persistent memory with the coordinator, so findings from earlier invocations are available automatically.

E. A subagent automatically receives every tool the coordinator has, regardless of its own definition.

F. Emitting several Task calls in a single coordinator response lets independent subagents run in parallel.

Question 30

Select ONE answer.

In a hub-and-spoke developer-productivity agent, the explorer subagent discovers a flaky test that the test-writer subagent should know about. How should this finding reach the test-writer?

A. The finding is left out because subagents should work independently without sharing findings.

B. The explorer returns it to the coordinator, which includes it in the test-writer's prompt.

C. The test-writer reads the explorer's conversation history on startup.

D. The explorer writes directly into the test-writer's context while both are running.

Question 31

Select ONE answer.

A coordinator's prompt lists nine numbered steps for the code explorer ('1. open src/, 2. read index.ts ...'). It works for one repository but fails in others with different layouts, and the subagent keeps following the steps even when they make no sense. What is the best redesign?

A. Remove all instructions so the subagent works only from its name and tool list.

B. Keep the steps but state that the subagent must follow them exactly, with no deviation permitted.

C. Add more numbered steps to cover each known repository layout and choose the right set of steps by looking at the repository name, updating the prompt when a new layout appears.

D. State the goal and quality criteria (entry points, key modules, external dependencies, with file references) and let the subagent decide how to explore.

Question 32

Select ONE answer.

A coordinator uses an iterative refinement loop: after each exploration round it checks whether key areas are covered and re-delegates gaps. The loop sometimes runs many rounds on obscure code without improving. What is the best way to bound it?

A. Define coverage criteria as the stopping condition, add a maximum number of rounds as a backstop, and report any remaining gaps explicitly.

B. Stop when the subagent's report becomes longer than the previous one, as growth indicates new content.

C. Stop after exactly two rounds in every case so that run time is always the same.

D. Let the loop continue until the coordinator feels it has explored enough, with no cap.

Question 33

Select ONE answer.

The explorer subagent finds the signature, file path, and behavior of 12 functions. The coordinator then spawns a test-writer subagent with the prompt 'Write tests for the billing functions found earlier.' The test-writer invents function names. What went wrong and what is the fix?

A. The test-writer's model is not strong enough for this task; replace it with a larger and more capable model while keeping exactly the same prompt, and rerun the job.

B. The test-writer has no access to the explorer's findings; include the complete findings (signatures, paths, behaviors) in structured form directly in its prompt.

C. The explorer should run again with the test-writer attached so that it can overhear the results.

D. The prompt is too long; shorten it to 'Write billing tests' so the subagent is not confused.

Question 34

Select ONE answer.

A developer-productivity agent is connected to three MCP servers that together expose 42 tools. The agent increasingly picks the wrong tool for ordinary requests. What is the best structural fix?

A. Switch tool_choice to forced selection on the most frequently used tool.

B. Keep all 42 tools and add a prompt table listing which tool to use for each of the 200 most common requests, and update the table every month.

C. Rename all 42 tools with a common prefix so that the model can group them more easily.

D. Give each agent or subagent only the small set of tools relevant to its role (roughly 4-5), keeping the rest out of its available tools.

Question 35

Select ONE answer.

You want the agent to inspect repository history and run linters without being able to push, delete, or rewrite anything. Which approach to Bash access is best?

A. Grant general Bash but run it only on Fridays when fewer developers are working.

B. Restrict shell access to a narrow set of read-only commands (such as git log, git diff, and the linter), rather than granting general Bash.

C. Grant general Bash and tell the agent in the prompt which commands it is allowed to run and which ones it must never run, with examples of both.

D. Remove Bash and have the developer paste the output of any command that the agent needs.

Question 36

Select ONE answer.

The agent keeps asking engineers which tables and columns exist before writing queries, or guesses column names and fails. The database schema is stable and well documented. How can you give the agent visibility without repeated exploratory calls?

A. Expose the database schema as an MCP resource so the agent can see the available tables and columns.

B. Paste the full schema into every user message to make sure the agent always has it.

C. Add a run_any_query tool and let the agent discover the schema by issuing exploratory queries each session.

D. Tell the agent to infer the schema from the names of the application's model classes.

Question 37

Select ONE answer.

A developer uses plan mode for every task, including adding a new CRUD endpoint that follows an existing, identical pattern in the same module, and finds the process slow. What is the better practice?

A. Ask a teammate to write the endpoint manually, because the tool is not suitable for repetitive work.

B. Use direct execution for well-understood, pattern-following changes like this, and reserve plan mode for tasks with architectural decisions or unclear scope.

C. Keep using plan mode but make each plan longer and more detailed so that it covers every field, route, and test of the new endpoint, reviewed by a teammate.

D. Switch to direct execution for everything, including multi-service restructuring, to avoid delays.

Question 38

Select ONE answer.

Engineers repeatedly ask for a new REST resource consisting of a model, route, and test file, each time retyping a long prompt. They want a shared, reusable, on-demand workflow that asks for the resource name. What should they build?

A. A .mcp.json entry that runs a server producing the three files without involving Claude.

B. A project skill in .claude/skills/ with a SKILL.md describing the workflow and an argument-hint for the resource name.

C. A section in CLAUDE.md that describes the scaffolding steps so that Claude loads them at the start of every session.

D. A personal note in each engineer's ~/.claude/CLAUDE.md with the prompt text to copy and paste.

Question 39

Select ONE answer.

A developer is unfamiliar with websocket reconnection and asks Claude Code to 'make the client reconnect'. They expect Claude to know all the requirements. What approach best surfaces unconsidered concerns before any code is written?

A. Ask Claude to implement the simplest possible reconnect loop and test it manually in the developer's environment before adding anything else.

B. Ask Claude for three different implementations and pick the shortest one.

C. Ask Claude to write the code and then summarize what it chose not to handle.

D. Ask Claude to interview them first about backoff, message loss, authentication refresh, and failure modes before implementing.

Question 40

Select ONE answer.

A manager is nervous about letting an AI assistant loose in an unfamiliar, business-critical billing module. They want Claude to understand the code and propose changes for review before any files are modified. Which mode supports this?

A. Letting Claude write changes straight to the main branch and rely on tests to catch issues.

B. Direct execution with a request to 'be careful' and a promise to review the diff afterwards.

C. Plan mode, which allows safe exploration and design before changes are committed.

D. Direct execution inside a user-level CLAUDE.md that forbids edits on Fridays.

Question 41

Select ONE answer.

The legacy-code summaries the agent produces vary: some list risks, some don't, and locations are described inconsistently. Which addition to the prompt gives the most consistent, actionable format?

A. A word limit that applies equally to all summaries, regardless of module size.

B. A few examples of ideal summaries showing the format: location, issue, severity, and suggested next step.

C. An instruction to write summaries in the style of the most recent summary the agent produced.

D. A paragraph reminding the model to be consistent and to include everything a developer might want to know.

Question 42

Select ONE answer.

When extracting hidden dependencies from legacy code, the agent finds them when documented in a header comment but misses them when mentioned in inline notes or config files. Which change is most likely to help?

A. Restrict the task to files that have header comments, since they contain the clearest documentation.

B. Require the agent to list at least ten dependencies per file to prevent empty results.

C. Instruct the agent to search more thoroughly, to be careful when reading unusual places such as inline notes, and to double-check its results before finishing.

D. Add few-shot examples showing dependencies being extracted from several different styles: header comments, inline notes, and config files.

Question 43

Select ONE answer.

In a large codebase exploration, the main agent needs to know where all test files for the checkout module live. The result could be hundreds of paths and lots of file reading. How should the main agent protect its context?

A. Delegate the question to a subagent that finds the test files and returns a concise summary, so that the main agent keeps its context for coordination.

B. Ask the user to list the test files manually.

C. Have the main agent read every candidate test file itself so that it holds the full details and can answer follow-up questions later without delegating.

D. Skip the question and assume tests live next to the source files.

Scenario 6: Structured Data Extraction (Questions 44-60)

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JSON schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

Question 44

Select ONE answer.

The extraction schema includes a boolean validated that the model sets to true when it believes the record is correct. A pre-call hook on write_to_erp allows the write only when validated is true. Bad records still reach the ERP. What is the design flaw and fix?

A. The prompt should state more strongly that validated may be true only when every field has been checked.

B. The flag should be an integer from 1 to 10 instead of a boolean, and the hook should allow writes at 8 or above.

C. The hook should be removed because validation belongs in the prompt, not in the pipeline.

D. The hook trusts a model-attested flag; have the hook run the validation checks itself (schema and business rules) and allow the write only if they pass.

Question 45

Select ONE answer.

The OCR tool returns, for each page, a very large array of word-level confidence values and bounding boxes. Only the text and a page-level confidence are needed for extraction, but the arrays fill the context and slow down every call. What is the best fix?

A. A PostToolUse hook that trims each result to the text and page-level confidence before the model processes it, optionally storing the full payload elsewhere.

B. A prompt instruction telling the model to ignore the bounding boxes and word-level confidence values in the OCR output and to focus on the page text, which saves effort.

C. A larger context window so that the arrays fit comfortably across more pages.

D. A second model that summarizes the OCR output before the extraction agent reads it.

Question 46

Select ONE answer.

An extraction service handles invoices, receipts, and contracts, each with a different schema and validation rules. A single prompt tries to handle all of them and performs inconsistently. What decomposition is best?

A. A single longer prompt that includes all three schemas and all validation rules, with an instruction to apply the right ones.

B. Three separate services, one per type, with users choosing which to upload to.

C. A prompt chain: first classify the document type, then extract with a type-specific schema and prompt, then run validation.

D. An open-ended agent that invents a new plan for every document, including its own schema.

Question 47

Select ONE answer.

The ERP posting tool can fail in three ways: the ERP is temporarily unavailable, the vendor ID in the record is invalid, and the accounting period is closed for postings. Which handling matches each failure best?

A. Treat all three as transient failures and retry each of them up to five times with a pause, logging each attempt, before finally reporting a failure to the user.

B. Report all three to the agent as the same generic failure so it does not take wrong action.

C. Treat all three as business errors and route every failure to manual review immediately.

D. Unavailable: transient and retryable; invalid vendor ID: validation error, fix input and resubmit; closed period: business error, not retryable, route for review.

Question 48

Select ONE answer.

The system prompt says: 'For anything that contains a dollar amount, use extract_invoice.' The extraction agent now sends receipts and contracts with amounts to extract_invoice, even though extract_receipt and extract_contract have good descriptions. What is the most effective fix?

A. Remove the keyword-based rule from the system prompt so selection relies on the tool descriptions and their boundaries.

B. Delete extract_receipt and extract_contract so there is only one extraction tool.

C. Add a second rule saying 'use extract_receipt for anything containing the word receipt'.

D. Move the rule into the tool description of extract_invoice so that it is closer to the tool.

Question 49

Select ONE answer.

lookup_vendor returns an empty list both when the vendor is genuinely new and when the master-data service times out. The agent treats an empty list as 'new vendor' and creates a new record, producing duplicates of existing vendors. What is the best change?

A. Add a prompt line telling the agent to be careful about creating duplicate vendors.

B. Make create_vendor idempotent on name alone, so repeated creation merges all records with equal names.

C. Return a transient error when the lookup fails and an empty success only for a completed search with no match; create a vendor only on the latter.

D. Keep the single empty result but have the agent call lookup_vendor three times in a row before it decides to create a new vendor record, on the theory that a service outage would not last that long.

Question 50

Select ONE answer.

The extraction flow first needs to classify an unknown document among three extractors (extract_invoice, extract_receipt, extract_contract), and then always call enrich_vendor with the result. Which tool_choice settings best implement this?

A. Use 'any' for the classification-and-extraction request, then force enrich_vendor in the following request.

B. Force extract_invoice for the first request, then 'auto' for the enrichment request.

C. Use 'auto' for both requests and rely on the prompt to make the model call tools.

D. Force enrich_vendor in the first request, then use 'any' for extraction.

Question 51

Select ONE answer.

Some supplier invoices have no discount line. The schema requires discount_percent as a number, and the model fills in 0 for these documents. Later, finance cannot tell 'no discount' from 'discount not stated'. What is the better schema design?

A. Make discount_percent nullable and instruct the model to return null when the document does not state a discount, reserving 0 for an explicitly stated 0%.

B. Make it a string and allow values such as 'none', 'n/a', or 'unknown' in free text.

C. Remove the field from the schema, since discounts are sometimes missing.

D. Keep it required and add a note in the prompt asking the model for its best estimate of the discount whenever the document does not state one, rounded to a whole percent.

Question 52

Select ONE answer.

A validation step finds that a line item's quantity is a string ('two') instead of a number. For the retry request, what should be included to maximize the chance of a correct extraction?

A. The original document, the failed extraction, and the specific validation error describing the field and expected type.

B. The original request repeated unchanged, with a note to try again.

C. Only the validation error text, without the document, so that the model is not distracted by unrelated content and can focus on the field.

D. The failed extraction and a request to make it better, without specifying the error.

Question 53

Select exactly TWO answers (choose 2).

Tool definitions use strict schema enforcement, so every tool input is syntactically valid and schema-conformant. Select the TWO remaining problems that are still possible and require semantic validation.

A. A tool input that omits a required property defined in the schema.

B. A ship-to address placed in the bill-to field.

C. A response that is not parseable JSON because of a missing closing brace.

D. A response in which a number field contains an unquoted letter sequence.

E. Line item amounts that do not add up to the stated invoice total.

Question 54

Select ONE answer.

A product manager proposes moving all extraction to the Message Batches API to cut costs by 50%, including the interactive upload screen where a user waits for confirmation of the extracted fields. How should you respond?

A. Reject batches for the whole system because their results cannot be correlated with requests.

B. Approve moving everything to batch and show a progress bar and a friendly message while the user waits for the results to come back, since the 50% savings justify it.

C. Keep the interactive upload on the synchronous API because batches have no latency guarantee, and use batches only for non-blocking bulk work.

D. Approve moving everything to batch but add a fallback to synchronous calls if results take over a minute.

Question 55

Select ONE answer.

A developer asks Claude for extraction results by writing 'Return JSON with these fields' in the prompt, and parses the text. About 2% of responses contain extra prose or invalid JSON. What is the most reliable improvement?

A. Define an extraction tool whose input schema is the target JSON, and read the result from the tool_use block.

B. Ask the model to double-check that its JSON is valid before it responds.

C. Add an instruction to the prompt saying 'respond only with valid JSON and no other text'.

D. Parse the first brace-delimited region in the response with a lenient parser and discard the rest.

Question 56

Select ONE answer.

Your extraction SLA promises every document's result within 48 hours of arrival. You submit accumulated documents to the Message Batches API at 06:00 and 18:00 each day, and a batch can take up to 24 hours. Documents that fail validation are corrected and resubmitted in the next scheduled submission. Which statement is accurate?

A. First-pass results arrive within 36 hours at worst (up to 12 waiting plus 24 processing), but a resubmitted document can take up to 72 hours, so the SLA cannot be guaranteed for retries.

B. The 48-hour SLA is guaranteed for every document, because the worst case of 36 hours is below 48 and a resubmission is simply another run of the same process.

C. The SLA becomes fully guaranteed if you move to one submission per day, because each document then gets exactly one pass and the schedule is easier to operate.

D. The SLA is violated even for first-pass results, because up to 12 hours of waiting plus up to 24 hours of processing is more than the 24-hour processing limit.

Question 57

Select ONE answer.

An invoice's payment status is sometimes unclear from the document (for example, a stamp that is half cut off). The schema's payment_status enum allows only 'paid' and 'unpaid', so the model guesses. What change is best?

A. Remove the field and let reviewers read the document to determine status.

B. Make the field free text so that the model can describe whatever it sees.

C. Add an 'unclear' value to the enum so the model can flag ambiguity instead of guessing.

D. Default to 'unpaid' whenever the model is uncertain, so errors err on the cautious side.

Question 58

Select ONE answer.

A Pydantic validator checks that line item totals sum to the invoice total and raises an error with the computed and stated values. What is the best use of that error?

A. Include it in a follow-up request together with the document and the failed extraction so the model can correct or flag the discrepancy.

B. Discard it and silently replace the stated total with the computed sum so the record always validates.

C. Use it to reject the document permanently without any retry or review.

D. Log it and accept the record anyway, since the schema itself is valid.

Question 59

Select ONE answer.

The team's validation results: overall field accuracy 96%. By type: typed invoices 99% (80% of volume), receipts 98% (15%), handwritten notes 71% (5%). By field: total 99%, tax amount 88%. They want to cut human review. Which plan is most defensible?

A. Keep full review for everything until overall accuracy reaches 100%.

B. Automate high-confidence totals on typed invoices and receipts, keep review for handwritten notes and the tax field, and keep sampling the automated stream.

C. Automate all fields for all document types because overall accuracy of 96% exceeds the usual bar, and revisit the plan only if complaints arrive from finance.

D. Automate all handwritten notes first, since they are the smallest share of volume and therefore low risk.

Question 60

Select ONE answer.

A vendor re-sends an invoice by email after a first copy was uploaded earlier. The pipeline creates two ledger records for the same invoice number. What is the best design?

A. Keep both records and let finance net them out at month-end.

B. Ask the model to decide whether two documents are duplicates based on how similar they look.

C. Always keep the first record and discard the second document entirely, as it adds nothing new and keeping both would only create clutter in the system later.

D. Detect duplicates on a natural key (vendor, invoice number, date), keep one record, and link both source documents to it so provenance is preserved.

End of examination. Stop here, then check your answers using the key that follows.

5. Quick answer key and score tracker

Mark each question you answered correctly (all required selections, no wrong ones). Then complete the domain tracker below.

No.	Ans.	Domain	Scen.	Mine
1	**B**	D1	S1	
2	**C**	D1	S1	
3	**C**	D1	S1	
4	**A**	D1	S1	
5	**B**	D1	S1	
6	**C**	D1	S1	
7	**A**	D1	S1	
8	**B**	D2	S1	
9	**C**	D2	S1	
10	**B**	D2	S1	
11	**D**	D4	S1	
12	**B,D**	D5	S1	
13	**B,E**	D5	S1	
14	**B**	D5	S1	
15	**C**	D5	S1	
16	**C**	D1	S2	
17	**D**	D2	S2	
18	**D**	D3	S2	
19	**C**	D3	S2	
20	**B**	D3	S2	
21	**C**	D3	S2	
22	**D**	D3	S2	
23	**B**	D3	S2	
24	**B,C**	D3	S2	
25	**A**	D3	S2	
26	**D,E**	D4	S2	
27	**D**	D5	S2	
28	**D**	D5	S2	
29	**A,C,F**	D1	S4	
30	**B**	D1	S4	

No.	Ans.	Domain	Scen.	Mine
31	**D**	D1	S4	
32	**A**	D1	S4	
33	**B**	D1	S4	
34	**D**	D2	S4	
35	**B**	D2	S4	
36	**A**	D2	S4	
37	**B**	D3	S4	
38	**B**	D3	S4	
39	**D**	D3	S4	
40	**C**	D3	S4	
41	**B**	D4	S4	
42	**D**	D4	S4	
43	**A**	D5	S4	
44	**D**	D1	S6	
45	**A**	D1	S6	
46	**C**	D1	S6	
47	**D**	D2	S6	
48	**A**	D2	S6	
49	**C**	D2	S6	
50	**A**	D2	S6	
51	**A**	D4	S6	
52	**A**	D4	S6	
53	**B,E**	D4	S6	
54	**C**	D4	S6	
55	**A**	D4	S6	
56	**A**	D4	S6	
57	**C**	D4	S6	
58	**A**	D4	S6	
59	**B**	D5	S6	
60	**D**	D5	S6	

Domain-level score tracker

Domain	Questions	Number list	My correct	% correct
1. Agentic Architecture & Orchestration	16	1, 2, 3, 4, 5, 6, 7, 16, 29, 30, 31, 32, 33, 44, 45, 46		
2. Tool Design & MCP Integration	11	8, 9, 10, 17, 34, 35, 36, 47, 48, 49, 50		
3. Claude Code Configuration & Workflows	12	18, 19, 20, 21, 22, 23, 24, 25, 37, 38, 39, 40		
4. Prompt Engineering & Structured Output	12	11, 26, 41, 42, 51, 52, 53, 54, 55, 56, 57, 58		
5. Context Management & Reliability	9	12, 13, 14, 15, 27, 28, 43, 59, 60		
Total	**60**			

Interpretation: percent-correct here is a practice signal only. The real exam's pass/fail decision is based on a single scaled score (cut score 720 of 1,000); domain percentages on the real score report are informational.

6. Detailed answer key and explanations

Each entry gives the correct answer, why it is correct, why every other option is wrong, the concept tested, the detail in the scenario that decides the answer, and a production lesson.

Q1 | Domain 1: Agentic Architecture & Orchestration | Scenario 1: Customer Support Resolution Agent

Task statement 1.4 | Intermediate | Single answer

Correct answer: B - A gate that rejects process_refund unless a successful lookup_order result for that order ID exists in the session and the amount is within the refundable balance.

Why the correct answer is correct: A prerequisite gate in code can check not only that a prior step happened but that its result supports the requested action: here a real order lookup and an amount within the refundable balance. This provides deterministic compliance where prompt instructions have a non-zero failure rate.

Option-by-option:

A. **[INCORRECT]** A requirement that the agent writes a short justification for each refund in the tool call, which a reviewer can read in the logs the following day.
A justification is read after the money has moved and does not prevent over-refunds.

B. **[CORRECT]** A gate that rejects process_refund unless a successful lookup_order result for that order ID exists in the session and the amount is within the refundable balance.
Correct. It ties the side effect to verified data and enforces the limit in code.

C. **[INCORRECT]** A system prompt rule instructing the agent to look up the order, confirm the amount with the customer in the conversation, and only then issue the refund through the tool.
A prompt rule is followed probabilistically, and the confirmed amount may still be wrong.

D. **[INCORRECT]** A lower per-call maximum for process_refund so that any single mistaken amount stays small, with retries allowed when the refund is below the limit.
A cap limits damage per call but does not verify the order or the balance, and retries can add up.

Key concept: Programmatic prerequisites tied to verified data (Task 1.4, 1.5).
Exam reasoning: Refunds based on user-stated numbers rather than retrieved records show that the action is not anchored to a verified source.
Production lesson: Make side-effect tools validate against system-of-record data, not against what the model or user says.

Q2 | Domain 1: Agentic Architecture & Orchestration | Scenario 1: Customer Support Resolution Agent

Task statement 1.4 | Challenging | Single answer

Correct answer: C - Because skipped verification exposes data and enables wrong financial actions, add a programmatic prerequisite gate rather than relying on prompts.

Why the correct answer is correct: When deterministic compliance is required, such as identity verification before financial or account operations, prompt instructions alone have a non-zero failure rate. Programmatic prerequisites block downstream calls until the prerequisite step is complete.

Option-by-option:

A. **[INCORRECT]** Keep adding examples until skipped verification is below 1%, then re-evaluate, since each new example should cut the failure rate further.
Prompting has diminishing returns and never reaches a guarantee.

B. **[INCORRECT]** Switch to a larger model, because a more capable model will follow the verification rule in every case.
Model capability does not turn a probabilistic behavior into a guarantee.

C. **[CORRECT]** Because skipped verification exposes data and enables wrong financial actions, add a programmatic prerequisite gate rather than relying on prompts.
Correct. Rules with financial or privacy consequences need deterministic enforcement.

D. **[INCORRECT]** Add an end-of-day report that lists conversations where verification was skipped so that a manager can contact the affected customers.
Detection after exposure does not prevent the harm and creates remediation work.

Key concept: Programmatic enforcement versus prompt guidance (Task 1.4).
Exam reasoning: The residual 4% after a prompt-only improvement is the key evidence, together with the sensitivity of the data.
Production lesson: Do not negotiate failure rates for rules that protect customers. Enforce them.

Q3 | Domain 1: Agentic Architecture & Orchestration | Scenario 1: Customer Support Resolution Agent

Task statement 1.1 | Intermediate | Single answer

Correct answer: C - The assistant's tool_use message and the matching tool_result must be appended to the conversation before the next request so the model can use the new information.

Why the correct answer is correct: In the agentic loop, tool results must be appended to conversation history so the model can reason about the next action. Omitting them makes each iteration start from the same state.

Option-by-option:

A. **[INCORRECT]** The model needs a stronger instruction not to repeat tool calls that it has already made earlier in the conversation.
From its point of view it has not made the call, because the history lacks it.

B. **[INCORRECT]** The tool should return its result as a system message so the model gives it higher priority on the next turn.
The standard mechanism is a tool_result in the conversation, not a system message.

C. **[CORRECT]** The assistant's tool_use message and the matching tool_result must be appended to the conversation before the next request so the model can use the new information.
Correct. Without the result in context, the model cannot move on and asks again.

D. **[INCORRECT]** The iteration cap is too low, and raising it to a much higher number will let the model eventually proceed without needing the earlier result at all, which is the cheapest fix.
More iterations simply repeat the same failing pattern.

Key concept: Appending tool results between iterations (Task 1.1).
Exam reasoning: A repeated identical call is the symptom of a model that never sees the answer to its earlier call.
Production lesson: If the model repeats itself, check what you sent it.

Q4 | Domain 1: Agentic Architecture & Orchestration | Scenario 1: Customer Support Resolution Agent

Task statement 1.4 | Intermediate | Single answer, secondary: D5

Correct answer: A - Customer ID C-2210; disputed charge $1,250 on order #7731 (duplicate billing on 2 Mar); root cause: payment captured twice; refund exceeds agent limit; recommended: reverse the duplicate capture; customer expects resolution within 5 days.

Why the correct answer is correct: Structured handoff summaries should include customer ID, root cause, amount, and recommended action, since the receiving human lacks access to the transcript. Including the customer's stated expectation helps preserve commitments.

Option-by-option:

A. **[CORRECT]** Customer ID C-2210; disputed charge $1,250 on order #7731 (duplicate billing on 2 Mar); root cause: payment captured twice; refund exceeds agent limit; recommended: reverse the duplicate capture; customer expects resolution within 5 days.
Correct. It contains identity, amount, cause, recommended action, and the customer's expectation.

B. **[INCORRECT]** Customer: Priya M. Issue: billing problem with a recent order. The customer is upset about a large amount and wants it fixed quickly. Please review the full chat for details.
It points to a transcript the specialist cannot see and omits IDs, amounts, and cause.

C. **[INCORRECT]** The complete transcript of all 42 messages and the raw JSON from every tool call, attached as a text file for the specialist to read.
It forces the specialist to reconstruct the case from noise.

D. **[INCORRECT]** Customer ID C-2210; order #7731; the customer is angry and has contacted us twice before; please handle with care and call them as soon as you can today.
It gives identifiers and sentiment but not the amount, root cause, or recommended action.

Key concept: Handoff content (Task 1.4, 5.1).
Exam reasoning: Only one payload contains the facts a specialist needs without further questions.
Production lesson: Write the handoff as if the reader has nothing else.

Q5 | Domain 1: Agentic Architecture & Orchestration | Scenario 1: Customer Support Resolution Agent

Task statement 1.4 | Challenging | Single answer

Correct answer: B - Make the check cumulative: evaluate the total refunded for the order including the new amount, and block and redirect when that total would exceed the threshold.

Why the correct answer is correct: Hooks that intercept tool calls can enforce policy deterministically, but the rule must be defined on the right quantity. Here the policy concerns total refunded amount, so state must be considered, not just the current call's argument.

Option-by-option:

A. **[INCORRECT]** Add a prompt instruction telling the agent never to split a refund into several smaller payments in order to avoid the review threshold, and to explain why.
Prompt instructions are probabilistic and this is a financial control.

B. **[CORRECT]** Make the check cumulative: evaluate the total refunded for the order including the new amount, and block and redirect when that total would exceed the threshold.
Correct. The rule's intent is a limit on total exposure, not per-call size.

C. **[INCORRECT]** Require the agent to wait 60 seconds between refund calls so that a person has time to notice unusual patterns.
A delay does not stop the total from exceeding the limit.

D. **[INCORRECT]** Reduce the per-call threshold to $250 so that splitting becomes less attractive to the agent.
A lower per-call limit can still be split into more calls and increases escalations unnecessarily.

Key concept: Tool call interception with correct rule semantics (Task 1.5, 1.4).
Exam reasoning: The $450 + $450 behavior bypasses a per-call check while violating its purpose.
Production lesson: Write guardrails against intent, not against a single argument.

Q6 | Domain 1: Agentic Architecture & Orchestration | Scenario 1: Customer Support Resolution Agent

Task statement 1.7 | Foundational | Single answer

Correct answer: C - Independent branches of a conversation created from a shared baseline, so different approaches can be explored without affecting each other.

Why the correct answer is correct: fork_session creates independent branches from a shared analysis baseline to explore divergent approaches. It is distinct from resuming a named session.

Option-by-option:

A. **[INCORRECT]** A faster version of the same conversation that runs with a cheaper model and discards earlier tool results to save time.
Forking does not change the model or discard history.

B. **[INCORRECT]** A way to run the same tool call in parallel across several customer accounts and merge the results into the main session.
Forking is about conversation branches, not fan-out of tool calls.

C. **[CORRECT]** Independent branches of a conversation created from a shared baseline, so different approaches can be explored without affecting each other.
Correct. That is the purpose of forking.

D. **[INCORRECT]** A permanent backup of the session transcript that is restored automatically if the agent crashes during a tool call.
That describes persistence or crash recovery, not forking.

Key concept: fork_session (Task 1.7).
Exam reasoning: Only one description matches branching from a shared baseline.
Production lesson: Know the session primitives: resume continues, fork branches.

Q7 | Domain 1: Agentic Architecture & Orchestration | Scenario 1: Customer Support Resolution Agent

Task statement 1.1 | Intermediate | Single answer

Correct answer: A - Let Claude choose tools within the loop because the right next action depends on context, and enforce the few non-negotiable rules with hooks and gates.

Why the correct answer is correct: Agentic loops are model-driven: Claude reasons about which tool to call next from context. Pre-configured decision trees are brittle for high-ambiguity requests, while hooks provide deterministic guarantees where needed.

Option-by-option:

A. **[CORRECT]** Let Claude choose tools within the loop because the right next action depends on context, and enforce the few non-negotiable rules with hooks and gates.
Correct. Model-driven decisions handle ambiguity, and hooks cover deterministic needs.

B. **[INCORRECT]** Let Claude choose tools and rely on the system prompt alone for rules such as refund limits and verification.
Prompt-only enforcement is unreliable for financial rules.

C. **[INCORRECT]** Use the decision tree for everything, because predictable sequences are always more reliable than model reasoning on ambiguous requests, and the team can test every path in advance.
A tree requires correct categorization first, which is the hard, ambiguous step.

D. **[INCORRECT]** Use the decision tree for category detection and the model only for writing the final reply to the customer.
This still depends on perfect categorization before any tool is chosen.

Key concept: Model-driven control with deterministic guardrails (Task 1.1, 1.5).
Exam reasoning: Ambiguity favors model-driven choice, while financial rules favor enforcement in code.
Production lesson: Let the model navigate and let code guard the cliffs.

Q8 | Domain 2: Tool Design & MCP Integration | Scenario 1: Customer Support Resolution Agent

Task statement 2.3 | Intermediate | Single answer

Correct answer: B - Use 'auto' for normal turns so the model can ask questions, and force a specific tool only at a step where it must run first and its inputs are available.

Why the correct answer is correct: tool_choice 'auto' lets the model respond with text or call a tool, 'any' forces some tool call, and forced selection names a tool. Forcing is appropriate when a particular call must happen and its inputs exist.

Option-by-option:

A. **[INCORRECT]** Keep 'any' on every turn and add a tool named ask_customer that wraps clarifying questions as a tool call, so that every turn still produces a tool call.
This adds an artificial tool and still forces a call every turn, including turns where no tool is needed.

B. **[CORRECT]** Use 'auto' for normal turns so the model can ask questions, and force a specific tool only at a step where it must run first and its inputs are available.
Correct. Forcing should be limited to where a call is required and possible.

C. **[INCORRECT]** Remove tool_choice entirely and tell the model in the prompt to always use a tool, never plain text.
Without a setting, nothing guarantees tool use and it still blocks clarifying questions.

D. **[INCORRECT]** Force get_customer on every turn so that identity lookup always happens before any reply.
Forcing every turn would call it repeatedly, even without an identifier.

Key concept: tool_choice trade-offs (Task 2.3).
Exam reasoning: Invented arguments are the cost of forcing calls when information is missing.
Production lesson: Constrain the model only where a constraint is both needed and satisfiable.

Q9 | Domain 2: Tool Design & MCP Integration | Scenario 1: Customer Support Resolution Agent

Task statement 2.1 | Foundational | Single answer

Correct answer: C - For each tool, the accepted input formats, example requests, and an explicit statement of when to use it versus the other tool.

Why the correct answer is correct: Tool descriptions are the primary mechanism for selection. Including input formats, example queries, edge cases, and boundaries relative to similar tools improves reliability.

Option-by-option:

A. **[INCORRECT]** A sentence at the end of each description saying that the tool is important and should be used whenever it might be relevant.
Importance statements encourage over-calling and add no distinguishing information.

B. **[INCORRECT]** A longer list of every field that the tool's backend database stores, whether or not the tool returns it.
Irrelevant fields add noise without clarifying purpose.

C. **[CORRECT]** For each tool, the accepted input formats, example requests, and an explicit statement of when to use it versus the other tool.
Correct. Boundaries and examples let the model distinguish similar tools.

D. **[INCORRECT]** The internal service name and team owner for each tool so that the model understands who maintains them.
Ownership metadata does not help decide which tool to call.

Key concept: Description content for selection (Task 2.1).
Exam reasoning: The problem is confusion between similar tools, which boundaries address.
Production lesson: Say what the tool is for, and what it is not for.

Q10 | Domain 2: Tool Design & MCP Integration | Scenario 1: Customer Support Resolution Agent

Task statement 2.4 | Intermediate | Single answer

Correct answer: B - Expose the catalog of return categories and policy versions as an MCP resource so the agent can see what exists without repeated tool calls.

Why the correct answer is correct: MCP resources expose content catalogs, such as documentation hierarchies and schemas, to give agents visibility into available data without exploratory tool calls.

Option-by-option:

A. **[INCORRECT]** Add three more search tools with different parameters and similar names so that the agent can find the right policy documents in fewer attempts.
More overlapping search tools add selection ambiguity.

B. **[CORRECT]** Expose the catalog of return categories and policy versions as an MCP resource so the agent can see what exists without repeated tool calls.
Correct. Resources give visibility into available content.

C. **[INCORRECT]** Ask the customer to state which policy applies so the agent does not have to look for it.
Customers don't know internal policy categories.

D. **[INCORRECT]** Remove search_policies and let the agent answer from its general knowledge of common return policies.
General knowledge does not reflect your company's policies.

Key concept: Resources for content catalogs (Task 2.4).
Exam reasoning: Repeated discovery calls for a stable catalog are what resources replace.
Production lesson: Publish catalogs as resources, keep tools for actions and queries.

Q11 | Domain 4: Prompt Engineering & Structured Output | Scenario 1: Customer Support Resolution Agent

Task statement 4.1 | Intermediate | Single answer

Correct answer: D - Provide explicit criteria with boundary rules, for example: refund already issued but charge still posted means billing dispute; item not yet returned means return.

Why the correct answer is correct: Explicit criteria and concrete examples reduce inconsistency better than general instructions, especially where categories overlap.

Option-by-option:

A. **[INCORRECT]** Ask the agent to handle every ambiguous message as both categories simultaneously and report two outcomes.
Dual handling creates conflicting actions and confusion.

B. **[INCORRECT]** Tell the agent to choose whichever category it thinks is more probable for each message, and to try to stay consistent in its choices across the whole day of conversations.
Without criteria, probability judgments stay inconsistent.

C. **[INCORRECT]** Require the customer to choose a category from a menu before the conversation starts.
Customers often do not know which category fits, which is why the request is ambiguous.

D. **[CORRECT]** Provide explicit criteria with boundary rules, for example: refund already issued but charge still posted means billing dispute; item not yet returned means return.
Correct. Explicit criteria resolve the overlap between categories.

Key concept: Explicit criteria for intent classification (Task 4.1).
Exam reasoning: The same message maps to different handling, which indicates an undefined boundary.
Production lesson: Define boundary cases in the prompt.

Q12 | Domain 5: Context Management & Reliability | Scenario 1: Customer Support Resolution Agent

Task statement 5.2 | Challenging | Select 2

Correct answer: B, D - The customer's request involves a situation the written policy does not cover or leaves ambiguous. / The agent cannot make meaningful progress after its tools keep failing or returning inconclusive results.

Why the correct answer is correct: Appropriate escalation triggers include policy exceptions or gaps and inability to make meaningful progress, in addition to explicit customer requests. Sentiment and self-reported confidence are unreliable proxies.

Option-by-option:

A. **[INCORRECT]** A sentiment score for the customer's last message falls below a set threshold.
Sentiment doesn't track whether the issue is beyond the agent's ability.

B. **[CORRECT]** The customer's request involves a situation the written policy does not cover or leaves ambiguous.
Correct. Policy gaps and exceptions need human decision-making.

C. **[INCORRECT]** The conversation has exceeded ten turns, regardless of progress.
Turn count is an arbitrary proxy for difficulty.

D. **[CORRECT]** The agent cannot make meaningful progress after its tools keep failing or returning inconclusive results.
Correct. Inability to progress is a legitimate trigger.

E. **[INCORRECT]** The model reports that its confidence in its answer is below 7 out of 10.
Self-reported confidence is poorly calibrated.

Key concept: Escalation triggers (Task 5.2).
Exam reasoning: The correct options reflect situations the agent cannot resolve; the wrong ones are indirect signals.
Production lesson: Escalate on capability limits, not on moods or counts.

Q13 | Domain 5: Context Management & Reliability | Scenario 1: Customer Support Resolution Agent

Task statement 5.2 | Intermediate | Select 2

Correct answer: B, E - When get_customer returns several accounts matching a name, ask for an additional identifier before proceeding. / When a customer explicitly asks for a human, escalate right away without first investigating the problem.

Why the correct answer is correct: Honor explicit human requests immediately, and ask for additional identifiers when tool results return multiple matches rather than selecting by heuristic.

Option-by-option:

A. **[INCORRECT]** When several accounts match, proceed with the one with the most recent activity and tell the customer which account was used.
Heuristic selection risks acting on the wrong account.

B. **[CORRECT]** When get_customer returns several accounts matching a name, ask for an additional identifier before proceeding.
Correct. Multiple matches require clarification instead of a guess.

C. **[INCORRECT]** When the agent feels unsure, ask the customer to rate how satisfied they are with the answers so far.
A satisfaction rating doesn't resolve uncertainty about the case.

D. **[INCORRECT]** When a customer sounds frustrated but has not asked for a person, escalate immediately to be safe.
If the issue is within the agent's ability, acknowledge and resolve.

E. **[CORRECT]** When a customer explicitly asks for a human, escalate right away without first investigating the problem.
Correct. Explicit requests are honored immediately.

Key concept: Escalation and ambiguity resolution (Task 5.2).
Exam reasoning: Each correct option is a specific rule from the guide.
Production lesson: Be literal about explicit requests and cautious about ambiguous identity.

Q14 | Domain 5: Context Management & Reliability | Scenario 1: Customer Support Resolution Agent

Task statement 5.1 | Challenging | Single answer

Correct answer: B - Keep a persistent case-facts block of amounts, dates, and customer expectations from tool results, included in every prompt outside the summarized history.

Why the correct answer is correct: Progressive summarization risks condensing numbers, dates, and customer-stated expectations into vague text. A persistent case-facts block extracted from tool results and kept outside summarized history preserves them.

Option-by-option:

A. **[INCORRECT]** Ask the model to re-derive the refund amount from the conversation each time it mentions it.
Re-deriving from a lossy summary reproduces the error.

B. **[CORRECT]** Keep a persistent case-facts block of amounts, dates, and customer expectations from tool results, included in every prompt outside the summarized history.
Correct. Facts are protected from summarization drift and sourced from authoritative data.

C. **[INCORRECT]** Switch to a model with a bigger context window so that no summarization is ever needed.
Larger windows delay but do not remove the need to protect key facts, and cost grows.

D. **[INCORRECT]** Summarize more often, say every five turns, so that each summary is closer in time to the facts it describes, and tell the summarizer to keep all numbers.
More summarization increases the chance of dropping specifics.

Key concept: Case facts and summarization risk (Task 5.1).
Exam reasoning: The summary lost both numeric values and timing, the exact items a facts block protects.
Production lesson: Don't summarize the numbers. Pin them.

Q15 | Domain 5: Context Management & Reliability | Scenario 1: Customer Support Resolution Agent

Task statement 5.2 | Intermediate | Single answer

Correct answer: C - Ask a short clarifying question to establish whether the customer means the order or the account before taking any action.

Why the correct answer is correct: Ambiguity resolution means asking for clarification when a request has multiple plausible meanings, especially when one action is irreversible.

Option-by-option:

A. **[INCORRECT]** Cancel the pending order, since it is the more likely meaning and the less destructive of the two.
A likely guess can still be wrong and the customer may be left with an unwanted state.

B. **[INCORRECT]** Escalate immediately because ambiguous cancellation requests are always too risky for an automated agent.
A single clarifying question usually resolves it, preserving first-contact resolution.

C. **[CORRECT]** Ask a short clarifying question to establish whether the customer means the order or the account before taking any action.
Correct. Ambiguity before an irreversible action calls for clarification.

D. **[INCORRECT]** Close the account and tell the customer how to reopen it if that was a mistake.
Closure is irreversible and shouldn't be done on a guess.

Key concept: Ambiguity resolution (Task 5.2).
Exam reasoning: 'Irreversible' raises the cost of guessing and makes clarification the cheapest safe step.
Production lesson: Ask before you act on an ambiguous, irreversible request.

Q16 | Domain 1: Agentic Architecture & Orchestration | Scenario 2: Code Generation with Claude Code

Task statement 1.7 | Intermediate | Single answer

Correct answer: C - Use fork_session to create two branches from the shared analysis and develop each strategy independently.

Why the correct answer is correct: fork_session creates independent branches from a shared analysis baseline to explore divergent approaches, such as comparing testing or refactoring strategies.

Option-by-option:

A. **[INCORRECT]** Ask Claude to implement both strategies in a single conversation, one after the other, and compare the results at the end.
The first strategy's context influences the second, and the conversation grows quickly.

B. **[INCORRECT]** Resume the same named session in two terminals and let each pursue a different strategy in the same history.
Both terminals continue one lineage, which defeats the independence you want.

C. **[CORRECT]** Use fork_session to create two branches from the shared analysis and develop each strategy independently.
Correct. Forked branches share the baseline and diverge without contaminating each other.

D. **[INCORRECT]** Start two brand-new sessions and ask each to redo the codebase analysis before building its strategy.
Repeating the analysis wastes effort and could produce two different baselines.

Key concept: fork_session usage (Task 1.7).
Exam reasoning: A shared starting analysis and two independent explorations are exactly the fork use case.
Production lesson: Branch when you need to compare, not when you need to continue.

Q17 | Domain 2: Tool Design & MCP Integration | Scenario 2: Code Generation with Claude Code

Task statement 2.5 | Intermediate | Single answer

Correct answer: D - Use Glob with a pattern such as **/*.stories.tsx to match file paths directly.

Why the correct answer is correct: Glob matches file path patterns (for example **/*.test.tsx), while Grep searches file contents.

Option-by-option:

A. **[INCORRECT]** Use Grep to search for the text 'stories' in every file and treat each matching file as a Storybook file.
Content matches would include unrelated files that mention the word.

B. **[INCORRECT]** Use Read on the root directory and ask Claude to scan the listing by eye for matching file names.
Reading is for file contents, and a manual scan of a large tree is impractical.

C. **[INCORRECT]** Use Bash to run find and cat on all files and then ask Claude to filter the combined output.
That produces large output and bypasses the dedicated tool.

D. **[CORRECT]** Use Glob with a pattern such as **/*.stories.tsx to match file paths directly.
Correct. Glob is designed for finding files by name patterns.

Key concept: Selecting Glob for name patterns (Task 2.5).
Exam reasoning: The target is defined by file name, not by file content.
Production lesson: Names with Glob, contents with Grep.

Q18 | Domain 3: Claude Code Configuration & Workflows | Scenario 2: Code Generation with Claude Code

Task statement 3.1 | Foundational | Single answer

Correct answer: D - Preferences in the user-level ~/.claude/CLAUDE.md, and team standards in the project-level CLAUDE.md committed to the repository.

Why the correct answer is correct: The CLAUDE.md hierarchy has user-level (personal, not shared), project-level (shared via version control), and directory-level scopes.

Option-by-option:

A. **[INCORRECT]** Both in the user-level ~/.claude/CLAUDE.md, so that the developer's settings follow them everywhere.
Team standards in a personal file are not shared with teammates.

B. **[INCORRECT]** Preferences in a directory-level CLAUDE.md in each package, and standards in the user-level file.
This reverses the scopes and scatters personal settings across the repository.

C. **[INCORRECT]** Both in the project-level CLAUDE.md, so that the whole team gets the same style of answers as well.
Personal preferences would be imposed on every teammate.

D. **[CORRECT]** Preferences in the user-level ~/.claude/CLAUDE.md, and team standards in the project-level CLAUDE.md committed to the repository.
Correct. User-level applies to one person; project-level is shared.

Key concept: CLAUDE.md scopes (Task 3.1).
Exam reasoning: One piece of guidance is personal and one is shared.
Production lesson: Scope instructions to the audience.

Q19 | Domain 3: Claude Code Configuration & Workflows | Scenario 2: Code Generation with Claude Code

Task statement 3.4 | Intermediate | Single answer

Correct answer: C - Use plan mode, have the Explore subagent do the search and return a summary of usage patterns, then design the migration and execute it.

Why the correct answer is correct: Plan mode enables safe exploration and design before committing to changes; the Explore subagent isolates verbose discovery output and returns summaries.

Option-by-option:

A. **[INCORRECT]** Run all searches in the main conversation and begin editing files as they appear in the results, fixing each usage as soon as it is found.
Verbose output fills the main context and editing before designing risks inconsistent changes.

B. **[INCORRECT]** Ask Claude to make the replacement in all 400 files in one direct-execution request with a one-line description.
Without design, edge cases and wrappers will be handled inconsistently.

C. **[CORRECT]** Use plan mode, have the Explore subagent do the search and return a summary of usage patterns, then design the migration and execute it.
Correct. Plan mode supports safe design, and Explore isolates the verbose discovery.

D. **[INCORRECT]** Skip the search and replace the API in the files the developer remembers using it.
Memory is incomplete for a large codebase.

Key concept: Plan mode with Explore (Task 3.4).
Exam reasoning: Scale and verbose search output together point to planning with isolation.
Production lesson: Isolate the noisy work, plan with the summary.

Q20 | Domain 3: Claude Code Configuration & Workflows | Scenario 2: Code Generation with Claude Code

Task statement 3.2 | Challenging | Single answer

Correct answer: B - context: fork, allowed-tools limited to Read and Write, and argument-hint: [pr-number].

Why the correct answer is correct: Skill frontmatter supports context: fork for isolation, allowed-tools for restricting tool access, and argument-hint for prompting required parameters.

Option-by-option:

A. **[INCORRECT]** context: fork, allowed-tools: Bash, Read, Write, and argument-hint: [pr-number].
Allowing Bash contradicts the requirement not to run shell commands.

B. **[CORRECT]** context: fork, allowed-tools limited to Read and Write, and argument-hint: [pr-number].
Correct. It isolates context, restricts tools, and prompts for the required argument.

C. **[INCORRECT]** context: fork, argument-hint: [pr-number], and a note in the body that Bash must not be used.
A note in the body does not restrict tools; allowed-tools does.

D. **[INCORRECT]** allowed-tools limited to Read and Write, argument-hint: [pr-number], and no context setting.
Without context: fork, the verbose output enters the main session.

Key concept: Skill frontmatter (Task 3.2).
Exam reasoning: Each requirement maps to one frontmatter field, and one distractor violates a requirement.
Production lesson: Match each requirement to a configuration field.

Q21 | Domain 3: Claude Code Configuration & Workflows | Scenario 2: Code Generation with Claude Code

Task statement 3.1 | Intermediate | Single answer

Correct answer: C - The team's rule that all new services must include structured logging and a health-check endpoint.

Why the correct answer is correct: Project-level CLAUDE.md holds shared standards committed to version control; user-level holds personal preferences that do not apply to teammates.

Option-by-option:

A. **[INCORRECT]** The developer's habit of asking Claude to open files in a particular editor on their machine.
This is machine-specific and personal.

B. **[INCORRECT]** A developer's preference for how long Claude's explanations are in their own terminal sessions.
Personal style preferences belong at the user level.

C. **[CORRECT]** The team's rule that all new services must include structured logging and a health-check endpoint.
Correct. Shared standards belong in the project-level file.

D. **[INCORRECT]** A reminder for the developer to review their own side projects on Friday afternoon.
This is unrelated to the shared repository.

Key concept: Memory scope (Task 3.1).
Exam reasoning: Only one item is a team standard.
Production lesson: Share rules, keep preferences personal.

Q22 | Domain 3: Claude Code Configuration & Workflows | Scenario 2: Code Generation with Claude Code

Task statement 3.3 | Intermediate | Single answer

Correct answer: D - Scope the rule with a glob by file type, such as paths: ["**/*.tf"], so it applies wherever .tf files are edited.

Why the correct answer is correct: Glob patterns in path-specific rules apply conventions by file type regardless of directory location.

Option-by-option:

A. **[INCORRECT]** Delete the paths field so the rule loads on every request and covers every directory.
Removing scoping adds irrelevant context to every session.

B. **[INCORRECT]** Move the Terraform conventions into the root CLAUDE.md so that they apply everywhere.
That loads Terraform conventions in unrelated work.

C. **[INCORRECT]** Duplicate the rule file for each infra/ subfolder with a path matching that subfolder.
Duplicated rules drift and miss new folders.

D. **[CORRECT]** Scope the rule with a glob by file type, such as paths: ["**/*.tf"], so it applies wherever .tf files are edited.
Correct. A file-type glob covers all locations.

Key concept: Glob-scoped rules (Task 3.3).
Exam reasoning: The files are defined by extension and live in several places.
Production lesson: Use the glob that matches the real category.

Q23 | Domain 3: Claude Code Configuration & Workflows | Scenario 2: Code Generation with Claude Code

Task statement 3.4 | Challenging | Single answer

Correct answer: B - Use plan mode for large, multi-file, or architectural tasks with several valid approaches, and direct execution for well-scoped changes with a clear cause.

Why the correct answer is correct: Plan mode is designed for complex tasks involving large-scale changes, multiple approaches, architectural decisions, and multi-file modifications; direct execution suits simple, well-scoped changes.

Option-by-option:

A. **[INCORRECT]** Use plan mode for every task over ten lines of code, since planning time is always cheaper than any rework, and the extra step guards against surprises.
A line count is not the criterion, and trivial tasks do not benefit from planning.

B. **[CORRECT]** Use plan mode for large, multi-file, or architectural tasks with several valid approaches, and direct execution for well-scoped changes with a clear cause.
Correct. Choose based on complexity and uncertainty.

C. **[INCORRECT]** Use plan mode only when a task touches more than one file, and never for changes inside a single file.
File count alone is not the criterion: a mechanical change repeated across files can be direct execution, and a single-file change can still need planning.

D. **[INCORRECT]** Use direct execution for all tasks and switch to plan mode only when the first attempt fails twice.
Architectural tasks should be planned before costly attempts.

Key concept: Mode selection criteria (Task 3.4).
Exam reasoning: Both extreme positions ignore task characteristics.
Production lesson: Match the process to the uncertainty of the task.

Q24 | Domain 3: Claude Code Configuration & Workflows | Scenario 2: Code Generation with Claude Code

Task statement 3.1 | Foundational | Select 2

Correct answer: B, C - Instructions in the project-level CLAUDE.md are shared with teammates through version control. / Instructions in ~/.claude/CLAUDE.md apply only to that user and are not shared with teammates.

Why the correct answer is correct: Project-level configuration (CLAUDE.md, .claude/commands, .claude/skills, .mcp.json) is shared via version control; user-level configuration (~/.claude/...) is personal.

Option-by-option:

A. **[INCORRECT]** Skills in ~/.claude/skills/ are shared with the team whenever the project is pulled.
User-level skills are personal.

B. **[CORRECT]** Instructions in the project-level CLAUDE.md are shared with teammates through version control.
Correct. Committed project files reach everyone who clones.

C. **[CORRECT]** Instructions in ~/.claude/CLAUDE.md apply only to that user and are not shared with teammates.
Correct. User-level files are personal.

D. **[INCORRECT]** MCP servers defined in a developer's ~/.claude.json are automatically available to every teammate.
User-scoped configuration is personal.

E. **[INCORRECT]** Slash commands in ~/.claude/commands/ are automatically available to everyone who clones the repository.
User-scoped commands are personal.

Key concept: Configuration scope (Task 3.1, 3.2).
Exam reasoning: One pair of statements follows the project/user distinction and the other pair reverses it.
Production lesson: If it lives in your home directory, it is yours alone.

Q25 | Domain 3: Claude Code Configuration & Workflows | Scenario 2: Code Generation with Claude Code

Task statement 3.3 | Challenging | Single answer

Correct answer: A - An imported file loads along with CLAUDE.md regardless of the file being edited, whereas a path-scoped rule loads only when matching test files are involved.

Why the correct answer is correct: @import keeps CLAUDE.md modular, but imported content loads with the file. Path-specific rules load conditionally when editing matching files, reducing irrelevant context.

Option-by-option:

A. **[CORRECT]** An imported file loads along with CLAUDE.md regardless of the file being edited, whereas a path-scoped rule loads only when matching test files are involved.
Correct. Conditional loading is the point of the path-scoped rule.

B. **[INCORRECT]** Path-scoped rules can contain more text than imported files, so the test conventions fit in one place.
Size is not the difference.

C. **[INCORRECT]** Path-scoped rules are applied to teammates only if they run a slash command first.
Rules load automatically based on file paths.

D. **[INCORRECT]** Imported files are ignored by Claude Code when they are referenced from the root CLAUDE.md, so conventions placed there would never take effect in any session.
@import is a supported way to modularize CLAUDE.md.

Key concept: Imports versus path-scoped rules (Task 3.1, 3.3).
Exam reasoning: The requirement to load only for test files distinguishes the two mechanisms.
Production lesson: Use modularity for organization and path scoping for relevance.

Q26 | Domain 4: Prompt Engineering & Structured Output | Scenario 2: Code Generation with Claude Code

Task statement 4.2 | Intermediate | Select 2

Correct answer: D, E - Include an ambiguous refactor-only example with a short explanation of why it is described the way it is. / Provide two to four examples of well-formed descriptions showing the desired sections (summary, risk, test plan).

Why the correct answer is correct: A few targeted few-shot examples demonstrate desired output format and, when they include reasoning for ambiguous cases, help the model generalize judgment.

Option-by-option:

A. **[INCORRECT]** Ask Claude to produce each description twice and choose the longer one.
Length is not a quality criterion.

B. **[INCORRECT]** Provide thirty examples drawn from last year's PRs so that the model has seen as many variations as possible.
Large numbers of examples add tokens and can reinforce inconsistent styles.

C. **[INCORRECT]** Write the format requirements in capital letters at the top of the prompt for emphasis.
Emphasis does not demonstrate the format.

D. **[CORRECT]** Include an ambiguous refactor-only example with a short explanation of why it is described the way it is.
Correct. Reasoned examples teach handling of ambiguous cases.

E. **[CORRECT]** Provide two to four examples of well-formed descriptions showing the desired sections (summary, risk, test plan).
Correct. Examples demonstrate the output format.

Key concept: Few-shot examples (Task 4.2).
Exam reasoning: Format consistency and ambiguous-case handling each have a matching technique.
Production lesson: Use a few, well-chosen examples with reasoning.

Q27 | Domain 5: Context Management & Reliability | Scenario 2: Code Generation with Claude Code

Task statement 5.4 | Intermediate | Single answer

Correct answer: D - Key findings: specific class and file names, call paths, decisions made, and open questions.

Why the correct answer is correct: Scratchpad files record key findings to persist them across context boundaries, counteracting degradation in extended sessions.

Option-by-option:

A. **[INCORRECT]** General notes about typical architectures of similar systems, to give Claude background knowledge.

Generic patterns are what degraded sessions already fall back on.

B. **[INCORRECT]** A running word count and timestamps for each action so that progress can be measured.
These are not the findings that answer later questions.

C. **[INCORRECT]** The full raw output of every search and file read performed so far.
Raw output reintroduces the volume you are trying to avoid.

D. **[CORRECT]** Key findings: specific class and file names, call paths, decisions made, and open questions.
Correct. Specific, durable findings counter context degradation.

Key concept: Scratchpad contents (Task 5.4).
Exam reasoning: The purpose is to retain specifics, not volume or generalities.
Production lesson: Write down conclusions, not transcripts.

Q28 | Domain 5: Context Management & Reliability | Scenario 2: Code Generation with Claude Code

Task statement 5.1 | Intermediate | Single answer

Correct answer: D - Lead with a short summary of the failing test and the question, trim the log to relevant sections, and organize the rest under clear headers.

Why the correct answer is correct: Models may omit findings from the middle of long inputs. Place key findings at the beginning, trim verbose output, and use explicit section headers.

Option-by-option:

A. **[INCORRECT]** Paste the log in reverse order so the end of the file appears first.
Reversing keeps the failure buried and makes it harder to read.

B. **[INCORRECT]** Ask Claude to read the log slowly and carefully before answering.
Instructions about care do not change attention distribution.

C. **[INCORRECT]** Paste the log twice in the same message so that the failing assertion appears in two places and is therefore more likely to be noticed by the model.
Duplication increases length without surfacing the failure.

D. **[CORRECT]** Lead with a short summary of the failing test and the question, trim the log to relevant sections, and organize the rest under clear headers.
Correct. Key information up front and structure mitigate position effects.

Key concept: Position effects in long input (Task 5.1).
Exam reasoning: The failure sits in the middle of a very long paste.
Production lesson: Put what matters first and cut what doesn't.

Q29 | Domain 1: Agentic Architecture & Orchestration | Scenario 4: Developer Productivity with Claude

Task statement 1.3 | Intermediate | Select 3

Correct answer: A, C, F - The coordinator's allowedTools must include "Task" for it to be able to spawn subagents. / Context needed by a subagent must be provided explicitly in its prompt because it does not inherit the coordinator's history. / Emitting several Task calls in a single coordinator response lets independent subagents run in parallel.

Why the correct answer is correct: Subagents are spawned via the Task tool (requires allowedTools to include "Task"), have isolated context that must be populated explicitly, and can be run in parallel by emitting multiple Task calls in a single response.

Option-by-option:

A. **[CORRECT]** The coordinator's allowedTools must include "Task" for it to be able to spawn subagents.
Correct. The Task tool is the spawning mechanism.

B. **[INCORRECT]** Subagents can see the coordinator's full conversation history, so passing results in the prompt is optional.
They do not inherit conversation history.

C. **[CORRECT]** Context needed by a subagent must be provided explicitly in its prompt because it does not inherit the coordinator's history.
Correct. Subagents operate with isolated context.

D. **[INCORRECT]** Subagents share a persistent memory with the coordinator, so findings from earlier invocations are available automatically.
Subagents do not share memory between invocations.

E. **[INCORRECT]** A subagent automatically receives every tool the coordinator has, regardless of its own definition.
Tool access is set per AgentDefinition.

F. **[CORRECT]** Emitting several Task calls in a single coordinator response lets independent subagents run in parallel.
Correct. Parallel spawning uses multiple Task calls in one response.

Key concept: Subagent invocation mechanics (Task 1.3).
Exam reasoning: The three correct statements each describe a documented mechanism; the three incorrect ones assume implicit sharing.
Production lesson: Subagents are isolated by default. Wire up everything they need.

Q30 | Domain 1: Agentic Architecture & Orchestration | Scenario 4: Developer Productivity with Claude

Task statement 1.2 | Foundational | Single answer

Correct answer: B - The explorer returns it to the coordinator, which includes it in the test-writer's prompt.

Why the correct answer is correct: In a hub-and-spoke architecture, the coordinator manages all inter-subagent communication, information routing, and error handling.

Option-by-option:

A. **[INCORRECT]** The finding is left out because subagents should work independently without sharing findings.
Independence of context doesn't mean relevant findings shouldn't be passed on.

B. **[CORRECT]** The explorer returns it to the coordinator, which includes it in the test-writer's prompt.
Correct. The coordinator manages information routing between subagents.

C. **[INCORRECT]** The test-writer reads the explorer's conversation history on startup.
Subagents don't have access to each other's conversation history.

D. **[INCORRECT]** The explorer writes directly into the test-writer's context while both are running.
Subagents have isolated contexts and cannot write into each other's.

Key concept: Hub-and-spoke routing (Task 1.2).
Exam reasoning: Only the coordinator sits between subagents.
Production lesson: All traffic goes through the hub.

Q31 | Domain 1: Agentic Architecture & Orchestration | Scenario 4: Developer Productivity with Claude

Task statement 1.3 | Challenging | Single answer

Correct answer: D - State the goal and quality criteria (entry points, key modules, external dependencies, with file references) and let the subagent decide how to explore.

Why the correct answer is correct: Coordinator prompts should specify research or exploration goals and quality criteria rather than step-by-step procedures, enabling subagent adaptability.

Option-by-option:

A. **[INCORRECT]** Remove all instructions so the subagent works only from its name and tool list.
Without a goal and quality criteria the output is unpredictable.

B. **[INCORRECT]** Keep the steps but state that the subagent must follow them exactly, with no deviation permitted.
Strict adherence worsens failures when the steps don't fit.

C. **[INCORRECT]** Add more numbered steps to cover each known repository layout and choose the right set of steps by looking at the repository name, updating the prompt when a new layout appears.
A growing list of layouts is brittle and never complete.

D. **[CORRECT]** State the goal and quality criteria (entry points, key modules, external dependencies, with file references) and let the subagent decide how to explore.
Correct. Goal-oriented prompts allow adaptation to different structures.

Key concept: Goal-oriented delegation (Task 1.3).
Exam reasoning: The failure appears when layouts differ, which a procedure cannot anticipate.
Production lesson: Tell subagents what good looks like, not exactly how to get there.

Q32 | Domain 1: Agentic Architecture & Orchestration | Scenario 4: Developer Productivity with Claude

Task statement 1.2 | Intermediate | Single answer, secondary: D5

Correct answer: A - Define coverage criteria as the stopping condition, add a maximum number of rounds as a backstop, and report any remaining gaps explicitly.

Why the correct answer is correct: Iterative refinement evaluates synthesis for gaps and re-delegates until coverage is sufficient; practical systems also add limits and surface remaining gaps.

Option-by-option:

A. **[CORRECT]** Define coverage criteria as the stopping condition, add a maximum number of rounds as a backstop, and report any remaining gaps explicitly.

Correct. Criteria drive stopping, a cap protects against runaway loops, and gaps are disclosed.

B. **[INCORRECT]** Stop when the subagent's report becomes longer than the previous one, as growth indicates new content.
Length doesn't measure coverage.

C. **[INCORRECT]** Stop after exactly two rounds in every case so that run time is always the same.
A fixed count ignores whether coverage was achieved.

D. **[INCORRECT]** Let the loop continue until the coordinator feels it has explored enough, with no cap.
No bound risks unlimited cost on obscure code.

Key concept: Refinement loops with stop conditions (Task 1.2, 5.3).
Exam reasoning: The loop needs a meaningful objective and a safety limit.
Production lesson: Stop on a goal, bound by a budget, and say what's missing.

Q33 | Domain 1: Agentic Architecture & Orchestration | Scenario 4: Developer Productivity with Claude

Task statement 1.3 | Challenging | Single answer

Correct answer: B - The test-writer has no access to the explorer's findings; include the complete findings (signatures, paths, behaviors) in structured form directly in its prompt.

Why the correct answer is correct: Subagent context must be explicitly provided in the prompt; subagents do not inherit parent context or share memory. Include complete findings, ideally structured, in the subagent's prompt.

Option-by-option:

A. **[INCORRECT]** The test-writer's model is not strong enough for this task; replace it with a larger and more capable model while keeping exactly the same prompt, and rerun the job.
No model can use information it has not received.

B. **[CORRECT]** The test-writer has no access to the explorer's findings; include the complete findings (signatures, paths, behaviors) in structured form directly in its prompt.
Correct. Subagents must be given the context explicitly.

C. **[INCORRECT]** The explorer should run again with the test-writer attached so that it can overhear the results.
There is no overhearing, and subagents don't share memory.

D. **[INCORRECT]** The prompt is too long; shorten it to 'Write billing tests' so the subagent is not confused.
Removing information makes invention more likely.

Key concept: Context passing (Task 1.3).
Exam reasoning: Invented names are the result of missing facts.
Production lesson: A subagent knows only what it is told.

Q34 | Domain 2: Tool Design & MCP Integration | Scenario 4: Developer Productivity with Claude

Task statement 2.3 | Intermediate | Single answer

Correct answer: D - Give each agent or subagent only the small set of tools relevant to its role (roughly 4-5), keeping the rest out of its available tools.

Why the correct answer is correct: Giving an agent too many tools degrades selection reliability. Scoped tool access, giving agents only the tools they need, improves it.

Option-by-option:

A. **[INCORRECT]** Switch tool_choice to forced selection on the most frequently used tool.
Forcing one tool prevents appropriate selection.

B. **[INCORRECT]** Keep all 42 tools and add a prompt table listing which tool to use for each of the 200 most common requests, and update the table every month.
A large table is hard to maintain and doesn't reduce decision complexity.

C. **[INCORRECT]** Rename all 42 tools with a common prefix so that the model can group them more easily.
Prefixes don't reduce the number of options.

D. **[CORRECT]** Give each agent or subagent only the small set of tools relevant to its role (roughly 4-5), keeping the rest out of its available tools.
Correct. Fewer relevant tools reduce decision complexity.

Key concept: Scoped tool sets (Task 2.3).
Exam reasoning: 42 tools is far above the 4-5 range called out as effective.
Production lesson: Reduce choice to improve choosing.

Q35 | Domain 2: Tool Design & MCP Integration | Scenario 4: Developer Productivity with Claude

Task statement 2.5 | Challenging | Single answer

Correct answer: B - Restrict shell access to a narrow set of read-only commands (such as git log, git diff, and the linter), rather than granting general Bash.

Why the correct answer is correct: Shell commands are high-risk tools. Scope them to the operations required, favoring read-only access where writes are not needed.

Option-by-option:

A. **[INCORRECT]** Grant general Bash but run it only on Fridays when fewer developers are working.
Timing does not control which commands run.

B. **[CORRECT]** Restrict shell access to a narrow set of read-only commands (such as git log, git diff, and the linter), rather than granting general Bash.
Correct. Least privilege limits the damage a mistaken command can do.

C. **[INCORRECT]** Grant general Bash and tell the agent in the prompt which commands it is allowed to run and which ones it must never run, with examples of both.
A prompt cannot prevent a destructive command from being issued.

D. **[INCORRECT]** Remove Bash and have the developer paste the output of any command that the agent needs.
This blocks the workflow when a narrow read-only grant would be safe.

Note: this item draws partly on **supplemental** engineering knowledge that is not a confirmed exam objective.
Key concept: Shell-command risk and least privilege (Task 2.5, 2.3). Supplemental practice beyond the guide's stated bullets.
Exam reasoning: The need is specific (read-only inspection), so access should be equally specific.
Production lesson: Grant the narrowest capability that does the job.

Q36 | Domain 2: Tool Design & MCP Integration | Scenario 4: Developer Productivity with Claude

Task statement 2.4 | Challenging | Single answer

Correct answer: A - Expose the database schema as an MCP resource so the agent can see the available tables and columns.

Why the correct answer is correct: MCP resources expose content catalogs such as database schemas to reduce exploratory tool calls.

Option-by-option:

A. **[CORRECT]** Expose the database schema as an MCP resource so the agent can see the available tables and columns.
Correct. Schemas are a named example of catalog content for resources.

B. **[INCORRECT]** Paste the full schema into every user message to make sure the agent always has it.
This wastes tokens and requires users to do it each time.

C. **[INCORRECT]** Add a run_any_query tool and let the agent discover the schema by issuing exploratory queries each session.
That creates the exploratory calls you wanted to avoid and gives broad access.

D. **[INCORRECT]** Tell the agent to infer the schema from the names of the application's model classes.
Inference can diverge from the real schema.

Key concept: Resources for schemas (Task 2.4).
Exam reasoning: The schema is stable reference data, a good fit for a resource.
Production lesson: Give agents a map, so they don't have to explore.

Q37 | Domain 3: Claude Code Configuration & Workflows | Scenario 4: Developer Productivity with Claude

Task statement 3.4 | Intermediate | Single answer

Correct answer: B - Use direct execution for well-understood, pattern-following changes like this, and reserve plan mode for tasks with architectural decisions or unclear scope.

Why the correct answer is correct: Direct execution is appropriate for simple, well-scoped changes; plan mode is for complex tasks with architectural implications.

Option-by-option:

A. **[INCORRECT]** Ask a teammate to write the endpoint manually, because the tool is not suitable for repetitive work.
Repetitive, pattern-based work is a good fit for direct execution.

B. **[CORRECT]** Use direct execution for well-understood, pattern-following changes like this, and reserve plan mode for tasks with architectural decisions or unclear scope.

Correct. Match the mode to the complexity.

C. **[INCORRECT]** Keep using plan mode but make each plan longer and more detailed so that it covers every field, route, and test of the new endpoint, reviewed by a teammate.
Longer plans add overhead to a task that does not need one.

D. **[INCORRECT]** Switch to direct execution for everything, including multi-service restructuring, to avoid delays.
Large architectural changes still need planning.

Key concept: Mode selection (Task 3.4).
Exam reasoning: The task mirrors an existing pattern, which lowers uncertainty.
Production lesson: Don't plan the obvious.

Q38 | Domain 3: Claude Code Configuration & Workflows | Scenario 4: Developer Productivity with Claude

Task statement 3.2 | Intermediate | Single answer

Correct answer: B - A project skill in .claude/skills/ with a SKILL.md describing the workflow and an argument-hint for the resource name.

Why the correct answer is correct: Skills are for on-demand, task-specific workflows; CLAUDE.md is for always-loaded universal standards. Project-scoped skills are shared via version control.

Option-by-option:

A. **[INCORRECT]** A .mcp.json entry that runs a server producing the three files without involving Claude.
That changes the design, and MCP servers are for tool and resource access, not workflow instructions.

B. **[CORRECT]** A project skill in .claude/skills/ with a SKILL.md describing the workflow and an argument-hint for the resource name.
Correct. Skills fit on-demand, task-specific workflows.

C. **[INCORRECT]** A section in CLAUDE.md that describes the scaffolding steps so that Claude loads them at the start of every session.
Always-loaded content adds context to sessions that don't need it.

D. **[INCORRECT]** A personal note in each engineer's ~/.claude/CLAUDE.md with the prompt text to copy and paste.
That is personal, not shared, and still manual.

Key concept: Skills for repetitive workflows (Task 3.2).
Exam reasoning: The workflow is reusable, shared, and invoked on demand.
Production lesson: On-demand workflows belong in skills.

Q39 | Domain 3: Claude Code Configuration & Workflows | Scenario 4: Developer Productivity with Claude

Task statement 3.5 | Foundational | Single answer

Correct answer: D - Ask Claude to interview them first about backoff, message loss, authentication refresh, and failure modes before implementing.

Why the correct answer is correct: The interview pattern has Claude ask questions to surface design considerations before implementing in unfamiliar domains.

Option-by-option:

A. **[INCORRECT]** Ask Claude to implement the simplest possible reconnect loop and test it manually in the developer's environment before adding anything else.
Manual testing of a naive version won't reveal missing design concerns.

B. **[INCORRECT]** Ask Claude for three different implementations and pick the shortest one.
Length is not a quality criterion for resilience.

C. **[INCORRECT]** Ask Claude to write the code and then summarize what it chose not to handle.
Learning the omissions after implementation costs rework.

D. **[CORRECT]** Ask Claude to interview them first about backoff, message loss, authentication refresh, and failure modes before implementing.
Correct. The interview pattern reveals design considerations the developer hasn't anticipated.

Key concept: Interview pattern (Task 3.5).
Exam reasoning: Unfamiliarity with the domain is the trigger for an interview.
Production lesson: When you don't know what to ask for, ask Claude to ask you.

Q40 | Domain 3: Claude Code Configuration & Workflows | Scenario 4: Developer Productivity with Claude

Task statement 3.4 | Intermediate | Single answer

Correct answer: C - Plan mode, which allows safe exploration and design before changes are committed.

Why the correct answer is correct: Plan mode enables safe codebase exploration and design before committing to changes, preventing costly rework.

Option-by-option:

A. **[INCORRECT]** Letting Claude write changes straight to the main branch and rely on tests to catch issues.
Bypassing review in a critical module is risky.

B. **[INCORRECT]** Direct execution with a request to 'be careful' and a promise to review the diff afterwards.
Direct execution modifies files, and review comes after the changes.

C. **[CORRECT]** Plan mode, which allows safe exploration and design before changes are committed.
Correct. Plan mode separates investigation and design from modification.

D. **[INCORRECT]** Direct execution inside a user-level CLAUDE.md that forbids edits on Fridays.
A calendar rule is unrelated to the review need.

Key concept: Plan mode for safety (Task 3.4).
Exam reasoning: The requirement is design and review before modification.
Production lesson: Review the plan, then authorize the edit.

Q41 | Domain 4: Prompt Engineering & Structured Output | Scenario 4: Developer Productivity with Claude

Task statement 4.2 | Intermediate | Single answer

Correct answer: B - A few examples of ideal summaries showing the format: location, issue, severity, and suggested next step.

Why the correct answer is correct: Few-shot examples demonstrating the specific desired output format (location, issue, severity, suggested fix) achieve consistency.

Option-by-option:

A. **[INCORRECT]** A word limit that applies equally to all summaries, regardless of module size.
A uniform length cap doesn't produce consistent content.

B. **[CORRECT]** A few examples of ideal summaries showing the format: location, issue, severity, and suggested next step.
Correct. Examples demonstrating format produce consistent, actionable output.

C. **[INCORRECT]** An instruction to write summaries in the style of the most recent summary the agent produced.
The previous summary may itself be inconsistent.

D. **[INCORRECT]** A paragraph reminding the model to be consistent and to include everything a developer might want to know.
General reminders don't specify the format.

Key concept: Few-shot for format (Task 4.2).
Exam reasoning: The inconsistency is in format, which examples fix.
Production lesson: Demonstrate the shape you want.

Q42 | Domain 4: Prompt Engineering & Structured Output | Scenario 4: Developer Productivity with Claude

Task statement 4.2 | Intermediate | Single answer

Correct answer: D - Add few-shot examples showing dependencies being extracted from several different styles: header comments, inline notes, and config files.

Why the correct answer is correct: Few-shot examples demonstrating correct extraction from documents with varied formats address missed or empty extractions.

Option-by-option:

A. **[INCORRECT]** Restrict the task to files that have header comments, since they contain the clearest documentation.
This ignores dependencies documented elsewhere.

B. **[INCORRECT]** Require the agent to list at least ten dependencies per file to prevent empty results.
Forcing a minimum encourages invented dependencies.

C. **[INCORRECT]** Instruct the agent to search more thoroughly, to be careful when reading unusual places such as inline notes, and to double-check its results before finishing.
General instructions don't show what to look for.

D. **[CORRECT]** Add few-shot examples showing dependencies being extracted from several different styles: header comments, inline notes, and config files.
Correct. Examples of varied structures improve extraction across formats.

Key concept: Few-shot for varied structures (Task 4.2).
Exam reasoning: Success depends on format, so the examples must cover formats.
Production lesson: Cover the variety you expect to see.

Q43 | Domain 5: Context Management & Reliability | Scenario 4: Developer Productivity with Claude

Task statement 5.4 | Foundational | Single answer

Correct answer: A - Delegate the question to a subagent that finds the test files and returns a concise summary, so that the main agent keeps its context for coordination.

Why the correct answer is correct: Subagent delegation isolates verbose exploration output, such as finding all test files, while the main agent preserves high-level coordination.

Option-by-option:

A. **[CORRECT]** Delegate the question to a subagent that finds the test files and returns a concise summary, so that the main agent keeps its context for coordination.
Correct. Subagents isolate verbose exploration.

B. **[INCORRECT]** Ask the user to list the test files manually.
This pushes tool-suited work onto the user.

C. **[INCORRECT]** Have the main agent read every candidate test file itself so that it holds the full details and can answer follow-up questions later without delegating.
That floods the main context.

D. **[INCORRECT]** Skip the question and assume tests live next to the source files.
An assumption could be wrong and cause missed coverage.

Key concept: Delegating exploration (Task 5.4).
Exam reasoning: A bulky discovery task with a small useful answer is ideal for a subagent.
Production lesson: Let helpers carry the weight, and bring back the answer.

Q44 | Domain 1: Agentic Architecture & Orchestration | Scenario 6: Structured Data Extraction

Task statement 1.5 | Intermediate | Single answer

Correct answer: D - The hook trusts a model-attested flag; have the hook run the validation checks itself (schema and business rules) and allow the write only if they pass.

Why the correct answer is correct: Hooks and gates provide deterministic guarantees only if they check facts themselves. A gate keyed to a flag the model sets inherits the model's unreliability.

Option-by-option:

A. **[INCORRECT]** The prompt should state more strongly that validated may be true only when every field has been checked.
Prompt wording doesn't turn a self-assertion into a verification.

B. **[INCORRECT]** The flag should be an integer from 1 to 10 instead of a boolean, and the hook should allow writes at 8 or above.
A numeric self-rating is still model-attested and poorly calibrated.

C. **[INCORRECT]** The hook should be removed because validation belongs in the prompt, not in the pipeline.
Removing the hook removes the only deterministic gate.

D. **[CORRECT]** The hook trusts a model-attested flag; have the hook run the validation checks itself (schema and business rules) and allow the write only if they pass.
Correct. The gate must verify, not rely on the model's claim.

Key concept: Deterministic enforcement that verifies (Task 1.5, 1.4).
Exam reasoning: Bad records pass because the gate checks a claim rather than the data.
Production lesson: Verify in code. Don't ask the model to vouch for itself.

Q45 | Domain 1: Agentic Architecture & Orchestration | Scenario 6: Structured Data Extraction

Task statement 1.5 | Intermediate | Single answer, secondary: D5

Correct answer: A - A PostToolUse hook that trims each result to the text and page-level confidence before the model processes it, optionally storing the full payload elsewhere.

Why the correct answer is correct: PostToolUse hooks intercept tool results for transformation before the model processes them, and trimming verbose outputs to relevant fields preserves context.

Option-by-option:

A. **[CORRECT]** A PostToolUse hook that trims each result to the text and page-level confidence before the model processes it, optionally storing the full payload elsewhere.
Correct. Hooks can transform results so only relevant fields reach the model.

B. **[INCORRECT]** A prompt instruction telling the model to ignore the bounding boxes and word-level confidence values in the OCR output and to focus on the page text, which saves effort.
The tokens are still consumed, and the model still sees the noise.

C. **[INCORRECT]** A larger context window so that the arrays fit comfortably across more pages.
More capacity does not remove cost or noise.

D. **[INCORRECT]** A second model that summarizes the OCR output before the extraction agent reads it.
This adds a stage and a possible loss when a deterministic trim suffices.

Key concept: Trimming with PostToolUse (Task 1.5, 5.1).
Exam reasoning: The needed fields are known, so deterministic trimming is appropriate.
Production lesson: Cut at the boundary, not inside the model.

Q46 | Domain 1: Agentic Architecture & Orchestration | Scenario 6: Structured Data Extraction

Task statement 1.6 | Foundational | Single answer

Correct answer: C - A prompt chain: first classify the document type, then extract with a type-specific schema and prompt, then run validation.

Why the correct answer is correct: Prompt chaining breaks predictable multi-step work into sequential focused steps, which fits classify, extract, and validate pipelines.

Option-by-option:

A. **[INCORRECT]** A single longer prompt that includes all three schemas and all validation rules, with an instruction to apply the right ones.
A single prompt still mixes concerns and increases inconsistency.

B. **[INCORRECT]** Three separate services, one per type, with users choosing which to upload to.
This pushes classification onto users who may not know the type.

C. **[CORRECT]** A prompt chain: first classify the document type, then extract with a type-specific schema and prompt, then run validation.
Correct. The stages are known in advance and each focuses on one job.

D. **[INCORRECT]** An open-ended agent that invents a new plan for every document, including its own schema.
Unbounded planning makes outputs and downstream integration unpredictable.

Key concept: Prompt chaining (Task 1.6).
Exam reasoning: The workflow has known stages with distinct needs.
Production lesson: Chain predictable steps.

Q47 | Domain 2: Tool Design & MCP Integration | Scenario 6: Structured Data Extraction

Task statement 2.2 | Intermediate | Single answer

Correct answer: D - Unavailable: transient and retryable; invalid vendor ID: validation error, fix input and resubmit; closed period: business error, not retryable, route for review.

Why the correct answer is correct: Distinguish transient errors (retry), validation errors (correct input), business errors (policy; non-retryable with explanation), and permission errors, using structured metadata.

Option-by-option:

A. **[INCORRECT]** Treat all three as transient failures and retry each of them up to five times with a pause, logging each attempt, before finally reporting a failure to the user.
Validation and business errors will never succeed on retry.

B. **[INCORRECT]** Report all three to the agent as the same generic failure so it does not take wrong action.
Generic failures prevent appropriate recovery.

C. **[INCORRECT]** Treat all three as business errors and route every failure to manual review immediately.
That overloads reviewers with failures a retry would resolve.

D. **[CORRECT]** Unavailable: transient and retryable; invalid vendor ID: validation error, fix input and resubmit; closed period: business error, not retryable, route for review.
Correct. Each category implies a different recovery.

Key concept: Error categories and retryability (Task 2.2).
Exam reasoning: Three different causes require three different responses.
Production lesson: Make the recovery obvious from the error.

Q48 | Domain 2: Tool Design & MCP Integration | Scenario 6: Structured Data Extraction

Task statement 2.1 | Intermediate | Single answer

Correct answer: A - Remove the keyword-based rule from the system prompt so selection relies on the tool descriptions and their boundaries.

Why the correct answer is correct: Keyword-sensitive instructions in a system prompt can create unintended tool associations that override well-written tool descriptions.

Option-by-option:

A. **[CORRECT]** Remove the keyword-based rule from the system prompt so selection relies on the tool descriptions and their boundaries.
Correct. Keyword-sensitive instructions can override good descriptions.

B. **[INCORRECT]** Delete extract_receipt and extract_contract so there is only one extraction tool.
This loses document-specific schemas.

C. **[INCORRECT]** Add a second rule saying 'use extract_receipt for anything containing the word receipt'.
More keyword rules create more overlaps.

D. **[INCORRECT]** Move the rule into the tool description of extract_invoice so that it is closer to the tool.
The rule would still bias selection toward extract_invoice.

Key concept: Prompt wording versus descriptions (Task 2.1).
Exam reasoning: The misrouting follows exactly the keyword in the prompt.
Production lesson: Audit prompts for rules that route by keyword.

Q49 | Domain 2: Tool Design & MCP Integration | Scenario 6: Structured Data Extraction

Task statement 2.2 | Challenging | Single answer

Correct answer: C - Return a transient error when the lookup fails and an empty success only for a completed search with no match; create a vendor only on the latter.

Why the correct answer is correct: Distinguish access failures (needing retry decisions) from valid empty results (successful queries with no matches) so the agent does not act on missing information.

Option-by-option:

A. **[INCORRECT]** Add a prompt line telling the agent to be careful about creating duplicate vendors.
The agent cannot see the difference between the two empty results.

B. **[INCORRECT]** Make create_vendor idempotent on name alone, so repeated creation merges all records with equal names.
Name-only merging would conflate different vendors and doesn't fix misread failures.

C. **[CORRECT]** Return a transient error when the lookup fails and an empty success only for a completed search with no match; create a vendor only on the latter.
Correct. Access failure and valid empty are distinguished, preventing duplicates.

D. **[INCORRECT]** Keep the single empty result but have the agent call lookup_vendor three times in a row before it decides to create a new vendor record, on the theory that a service outage would not last that long.
Repeated timeouts produce repeated empty results.

Key concept: Failure versus empty result (Task 2.2).
Exam reasoning: Duplicates arise when an outage is read as 'no match'.
Production lesson: Never let an outage look like an absence.

Q50 | Domain 2: Tool Design & MCP Integration | Scenario 6: Structured Data Extraction

Task statement 2.3 | Intermediate | Single answer, secondary: D4

Correct answer: A - Use 'any' for the classification-and-extraction request, then force enrich_vendor in the following request.

Why the correct answer is correct: tool_choice 'any' guarantees a tool call when the type is unknown; forced selection ensures a specific tool runs, with later steps in follow-up turns.

Option-by-option:

A. **[CORRECT]** Use 'any' for the classification-and-extraction request, then force enrich_vendor in the following request.
Correct. 'any' guarantees an extractor is called; forcing guarantees the enrichment step.

B. **[INCORRECT]** Force extract_invoice for the first request, then 'auto' for the enrichment request.
Forcing a single extractor would mishandle receipts and contracts.

C. **[INCORRECT]** Use 'auto' for both requests and rely on the prompt to make the model call tools.
'auto' permits text responses, so neither step is guaranteed.

D. **[INCORRECT]** Force enrich_vendor in the first request, then use 'any' for extraction.
Enrichment depends on the extraction result, so it cannot come first.

Key concept: Combining tool_choice modes (Task 2.3, 4.3).
Exam reasoning: The first step has unknown type and the second step is always the same tool.
Production lesson: Pick the guarantee that matches what you know at each step.

Q51 | Domain 4: Prompt Engineering & Structured Output | Scenario 6: Structured Data Extraction

Task statement 4.3 | Intermediate | Single answer

Correct answer: A - Make discount_percent nullable and instruct the model to return null when the document does not state a discount, reserving 0 for an explicitly stated 0%.

Why the correct answer is correct: Design schema fields as optional or nullable when sources may not contain the information, preventing the model from fabricating values to satisfy required fields.

Option-by-option:

A. **[CORRECT]** Make discount_percent nullable and instruct the model to return null when the document does not state a discount, reserving 0 for an explicitly stated 0%.
Correct. Null preserves the difference between unknown and zero.

B. **[INCORRECT]** Make it a string and allow values such as 'none', 'n/a', or 'unknown' in free text.
Free-form strings make downstream handling inconsistent.

C. **[INCORRECT]** Remove the field from the schema, since discounts are sometimes missing.
This loses real discounts that the documents do state.

D. **[INCORRECT]** Keep it required and add a note in the prompt asking the model for its best estimate of the discount whenever the document does not state one, rounded to a whole percent.
Asking for an estimate invents data the document never stated, which downstream systems would then treat as fact.

Key concept: Nullable fields (Task 4.3).
Exam reasoning: Zero and missing have different meanings, so the schema must separate them.
Production lesson: Represent 'unknown' honestly.

Q52 | Domain 4: Prompt Engineering & Structured Output | Scenario 6: Structured Data Extraction

Task statement 4.4 | Foundational | Single answer

Correct answer: A - The original document, the failed extraction, and the specific validation error describing the field and expected type.

Why the correct answer is correct: Retry-with-error-feedback includes the original document, the failed extraction, and specific validation errors to guide self-correction.

Option-by-option:

A. **[CORRECT]** The original document, the failed extraction, and the specific validation error describing the field and expected type.
Correct. This gives the model what it needs to self-correct.

B. **[INCORRECT]** The original request repeated unchanged, with a note to try again.
No new information is provided about what to fix.

C. **[INCORRECT]** Only the validation error text, without the document, so that the model is not distracted by unrelated content and can focus on the field.
Without the document the model cannot re-extract.

D. **[INCORRECT]** The failed extraction and a request to make it better, without specifying the error.
A vague request doesn't point to the defect.

Key concept: Retry with feedback (Task 4.4).
Exam reasoning: The model needs the source, the mistake, and the rule it broke.
Production lesson: Make retries informative.

Q53 | Domain 4: Prompt Engineering & Structured Output | Scenario 6: Structured Data Extraction

Task statement 4.4 | Challenging | Select 2

Correct answer: B, E - A ship-to address placed in the bill-to field. / Line item amounts that do not add up to the stated invoice total.

Why the correct answer is correct: Strict JSON schemas via tool use eliminate syntax errors but not semantic errors such as values that don't sum or values in wrong fields.

Option-by-option:

A. **[INCORRECT]** A tool input that omits a required property defined in the schema.
Schema enforcement via tool use prevents missing required properties.

B. **[CORRECT]** A ship-to address placed in the bill-to field.
Correct. A value in the wrong field is valid structurally but wrong.

C. **[INCORRECT]** A response that is not parseable JSON because of a missing closing brace.
Tool use with a schema eliminates JSON syntax errors.

D. **[INCORRECT]** A response in which a number field contains an unquoted letter sequence.
Type violations are structural errors that the schema prevents.

E. **[CORRECT]** Line item amounts that do not add up to the stated invoice total.
Correct. Valid JSON can contain inconsistent numbers.

Key concept: Syntax versus semantic errors (Task 4.4, 4.3).
Exam reasoning: Only two of the five problems are about meaning rather than shape.
Production lesson: Schemas fix form. Checks fix meaning.

Q54 | Domain 4: Prompt Engineering & Structured Output | Scenario 6: Structured Data Extraction

Task statement 4.5 | Intermediate | Single answer

Correct answer: C - Keep the interactive upload on the synchronous API because batches have no latency guarantee, and use batches only for non-blocking bulk work.

Why the correct answer is correct: The Message Batches API provides 50% savings with up to 24-hour processing and no latency SLA, so it suits latency-tolerant bulk work and not blocking, user-facing steps.

Option-by-option:

A. **[INCORRECT]** Reject batches for the whole system because their results cannot be correlated with requests.
Results can be correlated with custom_id.

B. **[INCORRECT]** Approve moving everything to batch and show a progress bar and a friendly message while the user waits for the results to come back, since the 50% savings justify it.
Batch completion can take up to 24 hours with no guarantee.

C. **[CORRECT]** Keep the interactive upload on the synchronous API because batches have no latency guarantee, and use batches only for non-blocking bulk work.
Correct. A waiting user needs predictable latency.

D. **[INCORRECT]** Approve moving everything to batch but add a fallback to synchronous calls if results take over a minute.
This adds complexity and the fallback would run most of the time.

Key concept: Batch suitability (Task 4.5).
Exam reasoning: A user waiting on screen needs a bounded response time.
Production lesson: Cheap is not the same as appropriate.

Q55 | Domain 4: Prompt Engineering & Structured Output | Scenario 6: Structured Data Extraction

Task statement 4.3 | Foundational | Single answer

Correct answer: A - Define an extraction tool whose input schema is the target JSON, and read the result from the tool_use block.

Why the correct answer is correct: Tool use with JSON schemas is the most reliable approach for guaranteed schema-compliant structured output, eliminating JSON syntax errors.

Option-by-option:

A. **[CORRECT]** Define an extraction tool whose input schema is the target JSON, and read the result from the tool_use block.
Correct. Tool use with a JSON schema is the most reliable path to schema-compliant output.

B. **[INCORRECT]** Ask the model to double-check that its JSON is valid before it responds.
A request to self-check is not enforcement.

C. **[INCORRECT]** Add an instruction to the prompt saying 'respond only with valid JSON and no other text'.
This helps but remains probabilistic.

D. **[INCORRECT]** Parse the first brace-delimited region in the response with a lenient parser and discard the rest.
Lenient parsing may mask or mis-parse malformed output.

Key concept: Structured output via tool_use (Task 4.3).
Exam reasoning: A small but real failure rate in prose-JSON points to enforcement.
Production lesson: Use the API's structure, not polite requests.

Q56 | Domain 4: Prompt Engineering & Structured Output | Scenario 6: Structured Data Extraction

Task statement 4.5 | Challenging | Single answer

Correct answer: A - First-pass results arrive within 36 hours at worst (up to 12 waiting plus 24 processing), but a resubmitted document can take up to 72 hours, so the SLA cannot be guaranteed for retries.

Why the correct answer is correct: Worst-case latency is the wait for the next submission slot plus the 24-hour processing window. A resubmission adds another wait and another window after the first result is known: up to 36 + 12 + 24 = 72 hours. Guarantees for retried documents therefore need a shorter interval, a retry budget in the SLA, or a different handling path for failures.

Option-by-option:

A. **[CORRECT]** First-pass results arrive within 36 hours at worst (up to 12 waiting plus 24 processing), but a resubmitted document can take up to 72 hours, so the SLA cannot be guaranteed for retries.
Correct. The retry repeats both the wait for the next slot and the processing window.

B. **[INCORRECT]** The 48-hour SLA is guaranteed for every document, because the worst case of 36 hours is below 48 and a resubmission is simply another run of the same process.
A resubmission restarts the wait and the processing window on top of the time already spent, so the total can exceed 48 hours.

C. **[INCORRECT]** The SLA becomes fully guaranteed if you move to one submission per day, because each document then gets exactly one pass and the schedule is easier to operate.
Failed documents still need a resubmission, and a daily schedule leaves no margin: the first pass alone can already take 48 hours.

D. **[INCORRECT]** The SLA is violated even for first-pass results, because up to 12 hours of waiting plus up to 24 hours of processing is more than the 24-hour processing limit.
The SLA is 48 hours, not 24. The first pass takes at most 36 hours, which is within it.

Key concept: SLA calculation for batches, including retry paths (Task 4.5).
Exam reasoning: The statement that failures wait for the next scheduled submission is the detail that breaks the guarantee for retried documents.
Production lesson: Compute SLAs from worst-case bounds and include the retry path, not just the first pass.

Q57 | Domain 4: Prompt Engineering & Structured Output | Scenario 6: Structured Data Extraction

Task statement 4.3 | Foundational | Single answer

Correct answer: C - Add an 'unclear' value to the enum so the model can flag ambiguity instead of guessing.

Why the correct answer is correct: Add enum values like 'unclear' for ambiguous cases, and 'other' plus detail fields for extensible categories.

Option-by-option:

A. **[INCORRECT]** Remove the field and let reviewers read the document to determine status.
That discards structured data for the many clear cases.

B. **[INCORRECT]** Make the field free text so that the model can describe whatever it sees.
Free text loses the categorical consistency downstream systems need.

C. **[CORRECT]** Add an 'unclear' value to the enum so the model can flag ambiguity instead of guessing.
Correct. An explicit escape value prevents forced guesses.

D. **[INCORRECT]** Default to 'unpaid' whenever the model is uncertain, so errors err on the cautious side.
A silent default misrepresents uncertainty as fact.

Key concept: Enum design with escape values (Task 4.3).
Exam reasoning: The model is forced to choose between two values that may both be wrong.
Production lesson: Give the model a way to say 'I can't tell'.

Q58 | Domain 4: Prompt Engineering & Structured Output | Scenario 6: Structured Data Extraction

Task statement 4.4 | Foundational | Single answer

Correct answer: A - Include it in a follow-up request together with the document and the failed extraction so the model can correct or flag the discrepancy.

Why the correct answer is correct: Semantic validation errors from tools like Pydantic should feed validation-retry loops, with specifics appended to the prompt on retry.

Option-by-option:

A. **[CORRECT]** Include it in a follow-up request together with the document and the failed extraction so the model can correct or flag the discrepancy.
Correct. Specific semantic validation errors guide self-correction.

B. **[INCORRECT]** Discard it and silently replace the stated total with the computed sum so the record always validates.
This hides a possible extraction or document error.

C. **[INCORRECT]** Use it to reject the document permanently without any retry or review.
Some discrepancies are fixable by retry or need human review.

D. **[INCORRECT]** Log it and accept the record anyway, since the schema itself is valid.
A known inconsistency should not pass silently.

Key concept: Pydantic validation and retry (Task 4.4).
Exam reasoning: The error contains the exact discrepancy the model needs.
Production lesson: Feed validation errors back to the model.

Q59 | Domain 5: Context Management & Reliability | Scenario 6: Structured Data Extraction

Task statement 5.5 | Challenging | Single answer

Correct answer: B - Automate high-confidence totals on typed invoices and receipts, keep review for handwritten notes and the tax field, and keep sampling the automated stream.

Why the correct answer is correct: Aggregate accuracy can mask poor performance on specific document types or fields. Validate accuracy by segment before automating, and keep stratified sampling for ongoing monitoring.

Option-by-option:

A. **[INCORRECT]** Keep full review for everything until overall accuracy reaches 100%.
This ignores segments where automation is already reliable.

B. **[CORRECT]** Automate high-confidence totals on typed invoices and receipts, keep review for handwritten notes and the tax field, and keep sampling the automated stream.
Correct. Automation is limited to segments with demonstrated accuracy, and sampling continues.

C. **[INCORRECT]** Automate all fields for all document types because overall accuracy of 96% exceeds the usual bar, and revisit the plan only if complaints arrive from finance.
The aggregate hides weak segments.

D. **[INCORRECT]** Automate all handwritten notes first, since they are the smallest share of volume and therefore low risk.
Small volume does not mean low error rate, and handwritten notes are the weakest segment.

Key concept: Segmented accuracy and automation decisions (Task 5.5).
Exam reasoning: The segment table shows large differences hidden by the average.
Production lesson: Automate where you've shown it works.

Q60 | Domain 5: Context Management & Reliability | Scenario 6: Structured Data Extraction

Task statement 5.6 | Intermediate | Single answer, secondary: D1

Correct answer: D - Detect duplicates on a natural key (vendor, invoice number, date), keep one record, and link both source documents to it so provenance is preserved.

Why the correct answer is correct: Duplicate handling should use deterministic keys, and provenance should be preserved by linking sources, keeping claim-source mappings intact.

Option-by-option:

A. **[INCORRECT]** Keep both records and let finance net them out at month-end.
Known duplicates would inflate balances until corrected.

B. **[INCORRECT]** Ask the model to decide whether two documents are duplicates based on how similar they look.
Similarity judgment is less reliable than a natural-key check.

C. **[INCORRECT]** Always keep the first record and discard the second document entirely, as it adds nothing new and keeping both would only create clutter in the system later.
Discarding the second document loses its provenance and any corrections it contains.

D. **[CORRECT]** Detect duplicates on a natural key (vendor, invoice number, date), keep one record, and link both source documents to it so provenance is preserved.
Correct. It avoids double-counting while retaining traceability.

Key concept: Duplicates with provenance (Task 5.6, 1.5).
Exam reasoning: A natural key identifies duplicates, and links preserve the evidence.
Production lesson: Deduplicate without losing where data came from.

---

7. Domain-level performance review guide

Use your tracker. A domain below roughly 70% on a paper deserves a focused review before the next paper. Look at the task statements and concepts listed for each domain and re-read the matching sections of the official Exam Guide.

Domain 1 - Agentic Architecture & Orchestration (27% official weight; 16 questions here)

Task	Topic	Questions
1.1	Agentic loops for autonomous task execution	3, 7
1.2	Coordinator-subagent orchestration	30, 32
1.3	Subagent invocation, context passing, spawning	29, 31, 33
1.4	Multi-step workflows: enforcement and handoff	1, 2, 4, 5
1.5	Agent SDK hooks: interception and normalization	44, 45
1.6	Task decomposition strategies	46
1.7	Session state, resumption, and forking	6, 16

Lessons to carry forward: Make side-effect tools validate against system-of-record data, not against what the model or user says. Do not negotiate failure rates for rules that protect customers. Enforce them. If the model repeats itself, check what you sent it. Write the handoff as if the reader has nothing else. Write guardrails against intent, not against a single argument. Know the session primitives: resume continues, fork branches.

Domain 2 - Tool Design & MCP Integration (18% official weight; 11 questions here)

Task	Topic	Questions
2.1	Tool interfaces: descriptions and boundaries	9, 48
2.2	Structured error responses for MCP tools	47, 49
2.3	Tool distribution and tool_choice	8, 34, 50
2.4	MCP server integration (Claude Code and agents)	10, 36
2.5	Built-in tools: Read, Write, Edit, Bash, Grep, Glob	17, 35

Lessons to carry forward: Constrain the model only where a constraint is both needed and satisfiable. Say what the tool is for, and what it is not for. Publish catalogs as resources, keep tools for actions and queries. Names with Glob, contents with Grep. Reduce choice to improve choosing. Grant the narrowest capability that does the job.

Domain 3 - Claude Code Configuration & Workflows (20% official weight; 12 questions here)

Task	Topic	Questions
3.1	CLAUDE.md hierarchy, scoping, modularity	18, 21, 24
3.2	Custom slash commands and skills	20, 38
3.3	Path-specific rules	22, 25
3.4	Plan mode vs direct execution	19, 23, 37, 40
3.5	Iterative refinement techniques	39

Lessons to carry forward: Scope instructions to the audience. Isolate the noisy work, plan with the summary. Match each requirement to a configuration field. Share rules, keep preferences personal. Use the glob that matches the real category. Match the process to the uncertainty of the task.

Domain 4 - Prompt Engineering & Structured Output (20% official weight; 12 questions here)

Task	Topic	Questions
4.1	Explicit criteria; reducing false positives	11
4.2	Few-shot prompting	26, 41, 42
4.3	Structured output via tool use and JSON schemas	51, 55, 57
4.4	Validation, retry, feedback loops	52, 53, 58

Task	Topic	Questions
4.5	Batch processing (Message Batches API)	54, 56

Lessons to carry forward: Define boundary cases in the prompt. Use a few, well-chosen examples with reasoning. Demonstrate the shape you want. Cover the variety you expect to see. Represent 'unknown' honestly. Make retries informative.

Domain 5 - Context Management & Reliability (15% official weight; 9 questions here)

Task	Topic	Questions
5.1	Context management across long interactions	14, 28
5.2	Escalation and ambiguity resolution	12, 13, 15
5.4	Context in large codebase exploration	27, 43
5.5	Human review workflows; confidence calibration	59
5.6	Provenance and uncertainty in multi-source synthesis	60

Lessons to carry forward: Escalate on capability limits, not on moods or counts. Be literal about explicit requests and cautious about ambiguous identity. Don't summarize the numbers. Pin them. Ask before you act on an ambiguous, irreversible request. Write down conclusions, not transcripts. Put what matters first and cut what doesn't.

8. Final revision checklist

Tick each item when you can explain it without looking at the key. Items are drawn from the concepts tested in this paper.

Done	Concept	First seen
[ ]	Programmatic prerequisites tied to verified data	Q1
[ ]	Programmatic enforcement versus prompt guidance	Q2
[ ]	Appending tool results between iterations	Q3
[ ]	Handoff content	Q4
[ ]	Tool call interception with correct rule semantics	Q5
[ ]	fork_session	Q6
[ ]	Model-driven control with deterministic guardrails	Q7
[ ]	tool_choice trade-offs	Q8
[ ]	Description content for selection	Q9
[ ]	Resources for content catalogs	Q10
[ ]	Explicit criteria for intent classification	Q11
[ ]	Escalation triggers	Q12
[ ]	Escalation and ambiguity resolution	Q13
[ ]	Case facts and summarization risk	Q14
[ ]	Ambiguity resolution	Q15
[ ]	fork_session usage	Q16
[ ]	Selecting Glob for name patterns	Q17
[ ]	CLAUDE.md scopes	Q18
[ ]	Plan mode with Explore	Q19
[ ]	Skill frontmatter	Q20
[ ]	Memory scope	Q21
[ ]	Glob-scoped rules	Q22
[ ]	Mode selection criteria	Q23
[ ]	Configuration scope	Q24
[ ]	Imports versus path-scoped rules	Q25
[ ]	Few-shot examples	Q26
[ ]	Scratchpad contents	Q27
[ ]	Position effects in long input	Q28
[ ]	Subagent invocation mechanics	Q29
[ ]	Hub-and-spoke routing	Q30
[ ]	Goal-oriented delegation	Q31
[ ]	Refinement loops with stop conditions	Q32
[ ]	Context passing	Q33
[ ]	Scoped tool sets	Q34
[ ]	Shell-command risk and least privilege	Q35
[ ]	Resources for schemas	Q36
[ ]	Mode selection	Q37
[ ]	Skills for repetitive workflows	Q38
[ ]	Interview pattern	Q39
[ ]	Plan mode for safety	Q40

Done	Concept	First seen
[ ]	Few-shot for format	Q41
[ ]	Few-shot for varied structures	Q42
[ ]	Delegating exploration	Q43
[ ]	Deterministic enforcement that verifies	Q44
[ ]	Trimming with PostToolUse	Q45
[ ]	Prompt chaining	Q46
[ ]	Error categories and retryability	Q47
[ ]	Prompt wording versus descriptions	Q48
[ ]	Failure versus empty result	Q49
[ ]	Combining tool_choice modes	Q50
[ ]	Nullable fields	Q51
[ ]	Retry with feedback	Q52
[ ]	Syntax versus semantic errors	Q53
[ ]	Batch suitability	Q54
[ ]	Structured output via tool_use	Q55
[ ]	SLA calculation for batches, including retry paths	Q56
[ ]	Enum design with escape values	Q57
[ ]	Pydantic validation and retry	Q58
[ ]	Segmented accuracy and automation decisions	Q59
[ ]	Duplicates with provenance	Q60

Before the next paper: classify each wrong answer as (a) knowledge gap, (b) misread of the stem or qualifier, or (c) a plausible-distractor trap. See CCAF_Final_Revision_Guide.pdf for the method.